# economics-finance: the Economics and Finance program, user guide

**economics-finance-ai999x 1.6.0**, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Economics and Finance covers time value of money, loans and annuities, bonds, stocks and options, portfolio theory, risk measures, actuarial mathematics, microeconomics, macroeconomics and econometrics. This guide is about `economics-finance`, the model as a self-contained program for Windows and Linux: 105 capabilities in 15 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/economics-finance-ai999x/); the Python package (`pip install economics-finance-ai999x`) has its own guide. Every command output printed here is what the program printed.

## Contents


- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability


## 1. Get it

Version 1.6.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

| what | system | download |
|---|---|---|
| Desktop app: Economics and Finance | Windows x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-economics-finance-1.6.0-windows-x86_64.zip |
| Desktop app: Economics and Finance | Linux x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-economics-finance-1.6.0-linux-x86_64.tar.gz |
| Desktop app: all models | Windows x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-windows-x86_64.zip |
| Desktop app: all models | Linux x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-linux-x86_64.tar.gz |
| Command line | Windows x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-windows-x86_64.zip |
| Command line | Linux x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-linux-x86_64.tar.gz |

### Desktop app

Economics and Finance in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-economics-finance-1.6.0-windows-x86_64.zip, right-click it → Extract All, then open the

folder and start ai999x-economics-finance.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-economics-finance-1.6.0-linux-
x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-economics-finance-1.6.0-linux-x86_64.tar.gz
./ai999x-economics-finance-1.6.0-linux-x86_64/ai999x-economics-finance
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.6.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.6.0-linux-x86_64.tar.gz
./ai999x-1.6.0-linux-x86_64/ai999x
```

### Command line, Linux

The bundle holds one program per model; `economics-finance` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.6.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.6.0-linux-x86_64:$PATH"' >> ~/.bashrc &&
source ~/.bashrc
economics-finance --version
```

### Command line, Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.6.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.6.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.6.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path",
"User") + ";$D", "User")
economics-finance --version
```

### In your browser

Open https://www.ai999x.com/models/economics-finance-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

### Python

```
pip install economics-finance-ai999x
```

## 2. Quick start

### 2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ economics-finance --version
economics-finance-ai999x 1.6.0 (Rust build, ax-engine 1.6.0)
```

### 2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ economics-finance ask "Find the monthly payment on a loan with principal = 200000, annual rate
= 6%, years = 30."
Read as: Loan payment (amortizing loan)
payment = 1199.10105
number_of_payments = 360
total_paid = 431676.3781
total_interest = 231676.3781
  by: Loan payment (amortizing loan)  (PMT = P i / (1 - (1 + i)^-n), i = annual rate / m, n =
years x m)
  annual_rate = 0.06
  principal = 200000
  years = 30
  assumes: principal > 0; 0 <= annual_rate < 1; years x payments_per_year a whole number, years
at most 1000
```

### 2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ economics-finance search "loan payment (amortizing loan)"
loan_payment                        Loan payment (amortizing loan)  (loans and annuities /
amortization)
loan_payment_split                  Interest and principal in a loan payment  (loans and
annuities / amortization)
number_of_periods                   Number of payments to repay a loan  (loans and annuities /
amortization)
loan_cumulative_interest            Cumulative interest of a loan  (loans and annuities /
amortization)
loan_balance                        Remaining balance of a loan  (loans and annuities /
amortization)
sinking_fund                        Sinking fund payment  (time value of money / annuities)
future_value_annuity                Future value of an annuity  (time value of money /
annuities)
present_value_annuity               Present value of an annuity  (time value of money /
annuities)
… 7 more
```

### 2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ economics-finance list
annuity_rate                          Interest rate of an annuity
arc_price_elasticity                  Price elasticity of demand (midpoint)
beta_regression                       Beta of an asset from return series
binomial_option                       Binomial option price (Cox-Ross-Rubinstein)
black_scholes                         Black-Scholes option prices
bond_convexity                        Convexity of a bond
bond_duration                         Macaulay and modified duration of a bond
bond_price                            Bond price from its yield
… 97 more
```

### 2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ economics-finance info loan_payment
Loan payment (amortizing loan)
  PMT = P i / (1 - (1 + i)^-n), i = annual rate / m, n = years x m
  in   principal amount borrowed [1]
  in   annual_rate nominal annual interest rate (decimal), compounded at the payment frequency
[1]
  in   years    term of the loan in years [1]
  in   payments_per_year payments per year (12 = monthly, default) (integer) [1]
  out  payment  level payment per period [1]
  out  number_of_payments number of payments n = years x m [1]
  out  total_paid payment x n [1]
  out  total_interest total paid - principal [1]
```

### 2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ economics-finance solve loan_payment principal=200000.0 annual_rate=0.06 years=30.0
payment = 1199.10105
number_of_payments = 360
total_paid = 431676.3781
total_interest = 231676.3781
  by: Loan payment (amortizing loan)  (PMT = P i / (1 - (1 + i)^-n), i = annual rate / m, n =
years x m)
  principal = 200000
  annual_rate = 0.06
  years = 30
  assumes: principal > 0; 0 <= annual_rate < 1; years x payments_per_year a whole number, years
at most 1000
```

### 2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ economics-finance solve loan_payment principal=200000.0 annual_rate=0.06 years=30.0 --json
{"outcome":"SOLVED","message":"","value":1199.1010503055138,"unit":"","solved_for":"payment","si":
[["principal",200000.0],["annual_rate",0.06],["years",30.0]],"values":
[["payment",1199.1010503055138,""],["number_of_payments",360,""],
["total_paid",431676.378109985,""],
["total_interest",231676.37810998497,""]],"target":"payment","phrase":"level payment per
period","assumptions":["principal > 0","0 <= annual_rate < 1","years x payments_per_year a whole
number, years at most 1000"],"name":"Loan payment (amortizing loan)","equation":"PMT = P i / (1 -
(1 + i)^-n), i = annual rate / m, n = years x m"}
```

### 2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ economics-finance solve loan_payment principal=200000.0 annual_rate=0.06
NEEDS_INPUT: loan_payment needs years (term of the loan in years). Example: solve('loan_payment',
principal=200000, annual_rate=0.06, years=30).
```

### 2.9 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | economics-finance batch

`cases.jsonl`:

```
{"key": "loan_payment", "inputs": {"principal": 200000.0, "annual_rate": 0.06, "years": 30.0}}
{"key": "irr", "inputs": {"cash_flows": "[-1000, 300, 400, 500]"}}
```

```
$ economics-finance batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":1199.1010503055138,"unit":"","solved_for":"payment","si":
[["principal",200000.0],["annual_rate",0.06],["years",30.0]],"values":
[["payment",1199.1010503055138,""],["number_of_payments",360,""],
["total_paid",431676.378109985,""],
["total_interest",231676.37810998497,""]],"target":"payment","phrase":"level payment per
period","assumptions":["principal > 0","0 <= annual_rate < 1","years x payments_per_year a whole
number, years at most 1000"],"name":"Loan payment (amortizing loan)","equation":"PMT = P i / (1 -
(1 + i)^-n), i = annual rate / m, n = years x m"}
{"outcome":"SOLVED","message":"","value":0.08896339469334996,"unit":"","solved_for":"irr","values":
[["irr",0.08896339469334996,""]],"target":"irr","phrase":"internal rate of return per
period","assumptions":["the cash flows change sign exactly once (otherwise the IRR is not unique
and none is given)"],"name":"Internal rate of return","equation":"bisection on v = 1/(1 + r) for
the unique positive root of sum CFt v^t, Newton polishing"}
```

## 3. Writing inputs

Inputs are written `name=value`, one per argument; `economics-finance info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

| kind | example | how it is read |
|---|---|---|
| a number with a unit | `"R=0.128 nm"` | any unit of the same kind; converted to SI |
| a number in SI | `R=1.28e-10` | no unit means the SI unit shown by info |

| kind | example | how it is read |
|---|---|---|
| a choice | `structure=FCC` | one of the values info lists |
| a list or vector | `"v=[1, 2, 3]" or "v=1, 2, 3"` | brackets keep it a list |
| a matrix | `"A=[[1, 2], [3, 4]]"` | one list per row |
| true / false | `inclusive=true` | for yes/no options |

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

## 4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

| outcome | meaning | exit status |
|---|---|---|
| SOLVED | the answer: every output with its unit, the method, the inputs in SI, the conditions assumed | 0 |
| NEEDS_INPUT | an input is missing; the message names it and shows a complete call | 1 |
| INVALID_INPUT | a value or unit of the wrong kind, an unknown input name, or a value outside the conditions | 1 |
| AMBIGUOUS | the inputs allow two different answers; say which one is meant | 1 |
| OUT_OF_SCOPE | the model has no capability for this, or the result would not be meaningful | 1 |
| (usage) | a command the program does not know: it prints its usage | 2 |

## 5. Scripts and agents: batch and JSON

`economics-finance solve ... --json` prints one answer as one line of JSON. `economics-finance batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | economics-finance batch`. `economics-finance catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

| answer field | meaning |
|---|---|
| outcome | one of the outcomes above |
| message | why there is no answer (empty when SOLVED) |
| value, unit | the main result in SI units |
| values | every output: [symbol, value, unit] (procedures) |
| target | the quantity solved for |
| si | each numeric input converted to SI: [symbol, value] |
| assumptions | the conditions the answer relies on |
| name, equation | the capability and its equation or method |

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["economics-finance", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

## 6. Figures

`economics-finance figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

## 7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `economics_finance_ai999x.wasm` (https://www.ai999x.com/compute/rust/economics_finance_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("economics_finance_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

## 8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `economics-finance info <id>` shows the same inputs and outputs in the terminal.

### Accounting (4)

#### Financial ratio analysis with the DuPont return on equity, `financial_ratios`

Liquidity (current, quick), solvency (debt ratio, debt to equity), efficiency (asset turnover) and profitability (net margin, ROA, ROE) ratios from balance-sheet and income-statement items, with ROE broken into its DuPont factors: margin x asset turnover x equity multiplier.

$$\mathrm{ROE} = \frac{NI}{E} = \frac{NI}{S} \cdot \frac{S}{TA} \cdot \frac{TA}{E}, \qquad \mathrm{current} = \frac{CA}{CL}, \quad \mathrm{quick} = \frac{CA - \mathrm{Inv}}{CL}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `sales` | in | net sales (revenue) of the period | , |
| `net_income` | in | net income (negative for a loss) | , |
| `total_assets` | in | total assets | , |
| `total_equity` | in | total shareholders' equity (positive, at most total assets) | , |
| `current_assets` | in | current assets | , |
| `current_liabilities` | in | current liabilities (positive) | , |
| `inventory` | in | inventory (part of current assets) | , |
| `current_ratio` | out | current assets / current liabilities | , |
| `quick_ratio` | out | (current assets - inventory) / current liabilities | , |
| `debt_ratio` | out | total liabilities / total assets | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `debt_to_equity` | out | total liabilities / total equity | , |
| `net_profit_margin` | out | net income / sales (DuPont factor 1) | , |
| `asset_turnover` | out | sales / total assets (DuPont factor 2) | , |
| `equity_multiplier` | out | total assets / total equity (DuPont factor 3) | , |
| `roa` | out | return on assets: net income / total assets | , |
| `roe` | out | return on equity: net income / equity = margin x turnover x multiplier | , |

**Applies when:** sales, total assets, equity, current liabilities > 0; equity <= total assets; inventory <= current assets <= total assets; current liabilities <= total assets - equity.

```
$ economics-finance solve financial_ratios sales=500000 net_income=40000 total_assets=400000
total_equity=160000 current_assets=150000 current_liabilities=100000 inventory=60000
current_ratio = 1.5
quick_ratio = 0.9
debt_ratio = 0.6
debt_to_equity = 1.5
net_profit_margin = 0.08
asset_turnover = 1.25
equity_multiplier = 2.5
roa = 0.1
roe = 0.25
  by: Financial ratio analysis with the DuPont return on equity  (ratios of the stated items;
liabilities = assets - equity; ROE = (NI/S)(S/TA)(TA/E))
  sales = 500000
  net_income = 40000
  total_assets = 400000
  total_equity = 160000
  current_assets = 150000
  current_liabilities = 100000
  inventory = 60000
  assumes: sales, total assets, equity, current liabilities > 0; equity <= total assets;
inventory <= current assets <= total assets; current liabilities <= total assets - equity
```

In the worked example, sales = 500000, net_income = 40000, total_assets = 400000, total_equity = 160000, current_assets = 150000, current_liabilities = 100000, inventory = 60000 go through the equation above and give current_ratio = 1.5, quick_ratio = 0.9, debt_ratio = 0.6, debt_to_equity = 1.5. It applies only when sales, total assets, equity, current liabilities > 0; equity <= total assets; inventory <= current assets <= total assets; current liabilities <= total assets - equity: outside that it says what is wrong instead of returning a number.

**Inventory costing: FIFO, LIFO and weighted average, `inventory_costing`**

Cost of goods sold and ending inventory under FIFO, LIFO and the weighted average cost (periodic system) from the cost layers of the period and the units sold, with the gross margin when a selling price is given.

$$\bar{c} = \frac{\sum_k u_k c_k}{\sum_k u_k}, \qquad \mathrm{COGS} + \mathrm{EI} = \sum_k u_k c_k$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `layer_units` | in | units in each cost layer, in order of acquisition (beginning inventory first) | , |
| `layer_costs` | in | unit cost of each layer, in the same order | , |
| `units_sold` | in | units sold in the period | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `selling_price` | in | selling price per unit (optional: gives sales revenue and gross margins) (optional) | , |
| `units_available` | out | units available for sale | , |
| `cost_of_goods_available` | out | cost of goods available for sale | , |
| `fifo_cogs` | out | cost of goods sold, FIFO (oldest layers sold first) | , |
| `fifo_ending_inventory` | out | ending inventory, FIFO (newest layers left) | , |
| `lifo_cogs` | out | cost of goods sold, LIFO (newest layers sold first) | , |
| `lifo_ending_inventory` | out | ending inventory, LIFO (oldest layers left) | , |
| `average_unit_cost` | out | weighted average unit cost: goods available / units available | , |
| `average_cogs` | out | cost of goods sold at the weighted average cost | , |
| `average_ending_inventory` | out | ending inventory at the weighted average cost | , |
| `sales_revenue` | out | selling price x units sold (with a selling price) | , |
| `fifo_gross_margin` | out | sales revenue - FIFO cost of goods sold | , |
| `lifo_gross_margin` | out | sales revenue - LIFO cost of goods sold | , |
| `average_gross_margin` | out | sales revenue - weighted average cost of goods sold | , |

**Applies when:** one unit cost per layer; units of each layer > 0; unit costs >= 0; 0 <= units sold <= units available; selling price >= 0.

```
$ economics-finance solve inventory_costing "layer_units=[100, 200, 150]" "layer_costs=[10, 12,
14]" units_sold=300 selling_price=20
units_available = 450
cost_of_goods_available = 5500
fifo_cogs = 3400
fifo_ending_inventory = 2100
lifo_cogs = 3900
lifo_ending_inventory = 1600
average_unit_cost = 12.22222222
average_cogs = 3666.666667
average_ending_inventory = 1833.333333
sales_revenue = 6000
fifo_gross_margin = 2600
lifo_gross_margin = 2100
average_gross_margin = 2333.333333
  by: Inventory costing: FIFO, LIFO and weighted average  (periodic system: FIFO takes the units
sold from the first layers, LIFO from the last; weighted average cost = sum(units x cost) /
sum(units); ending inventory = the units left at their costs)
  units_sold = 300
  selling_price = 20
  assumes: one unit cost per layer; units of each layer > 0; unit costs >= 0; 0 <= units sold <=
units available; selling price >= 0
```

In the worked example, layer_units = [100, 200, 150], layer_costs = [10, 12, 14], units_sold = 300, selling_price = 20 go through the equation above and give units_available = 450, cost_of_goods_available = 5500, fifo_cogs = 3400, fifo_ending_inventory = 2100. It applies only when one unit cost per layer; units of each layer > 0; unit costs >= 0; 0 <= units sold <= units available; selling price >= 0: outside that it says what is wrong instead of returning a number.

**Overhead variances: variable spending and efficiency, fixed budget and volume, `overhead_cost_variances`**

The four overhead variances of a standard costing system with overhead applied on hours: variable overhead spending and efficiency, fixed overhead budget (spending) and volume, and the total over- or under-applied overhead, each marked favourable or unfavourable.

$$\text{spending} = \text{AVOH} - AH \cdot VR, \quad \text{efficiency} = VR\,(AH - SH), \quad \text{volume} = \text{BFOH} - SH \cdot \frac{\text{BFOH}}{DH}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `variable_overhead_rate` | in | standard variable overhead rate per hour | , |
| `actual_variable_overhead` | in | actual variable overhead cost | , |
| `actual_hours` | in | actual hours of the allocation base | , |
| `standard_hours` | in | standard hours allowed for the actual output | , |
| `budgeted_fixed_overhead` | in | budgeted fixed overhead | , |
| `actual_fixed_overhead` | in | actual fixed overhead cost | , |
| `denominator_hours` | in | denominator hours on which the fixed overhead rate was set (positive) | , |
| `variable_spending_variance` | out | actual variable overhead - AH x VR | , |
| `variable_efficiency_variance` | out | VR (AH - SH) | , |
| `fixed_budget_variance` | out | actual fixed overhead - budgeted fixed overhead | , |
| `fixed_volume_variance` | out | budgeted fixed overhead - SH x FR | , |
| `total_overhead_variance` | out | actual overhead - applied overhead (sum of the four) | , |
| `fixed_overhead_rate` | out | FR = budgeted fixed overhead / denominator hours | , |
| `applied_overhead` | out | SH x (VR + FR) | , |
| `variable_spending_type` | out | favourable, unfavourable or none | , |
| `variable_efficiency_type` | out | favourable, unfavourable or none | , |
| `fixed_budget_type` | out | favourable, unfavourable or none | , |
| `fixed_volume_type` | out | favourable, unfavourable or none | , |
| `total_type` | out | favourable (over-applied), unfavourable (under-applied) or none | , |

**Applies when:** amounts, rates and hours >= 0; denominator hours > 0.

```
$ economics-finance solve overhead_cost_variances variable_overhead_rate=2
actual_variable_overhead=9000 actual_hours=4200 standard_hours=4000 budgeted_fixed_overhead=12000
actual_fixed_overhead=12500 denominator_hours=5000
variable_spending_variance = 600
variable_efficiency_variance = 400
fixed_budget_variance = 500
fixed_volume_variance = 2400
total_overhead_variance = 3900
fixed_overhead_rate = 2.4
applied_overhead = 17600
variable_spending_type = unfavourable
variable_efficiency_type = unfavourable
fixed_budget_type = unfavourable
fixed_volume_type = unfavourable
total_type = unfavourable
  by: Overhead variances: variable spending and efficiency, fixed budget and volume  (spending =
AVOH - AH VR; efficiency = VR (AH - SH); budget = AFOH - BFOH; volume = BFOH (DH - SH)/DH; total
= AVOH + AFOH - SH (VR + BFOH/DH))
  variable_overhead_rate = 2
  actual_variable_overhead = 9000
  actual_hours = 4200
  standard_hours = 4000
  budgeted_fixed_overhead = 12000
  actual_fixed_overhead = 12500
  denominator_hours = 5000
  assumes: amounts, rates and hours >= 0; denominator hours > 0
```

In the worked example, variable_overhead_rate = 2, actual_variable_overhead = 9000, actual_hours = 4200, standard_hours = 4000, budgeted_fixed_overhead = 12000, actual_fixed_overhead = 12500, denominator_hours = 5000 go through the equation above and give variable_spending_variance = 600, variable_efficiency_variance = 400, fixed_budget_variance = 500, fixed_volume_variance = 2400. It applies only when amounts, rates and hours >= 0; denominator hours > 0: outside that it says what is wrong instead of returning a number.

**Standard-cost variances: materials price and quantity, labour rate and efficiency, `direct_cost_variances`**

Price (labour: rate) and quantity (labour: efficiency) variances of direct materials or direct labour from standard and actual prices and quantities, each marked favourable or unfavourable.

$$PV = Q_P\,(AP - SP), \qquad QV = SP\,(AQ - SQ)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `standard_price` | in | standard price per unit of input (labour: standard wage rate per hour) | , |
| `standard_quantity` | in | standard quantity allowed for the actual output (labour: standard hours allowed) | , |
| `actual_price` | in | actual price paid per unit of input (labour: actual wage rate) | , |
| `actual_quantity` | in | actual quantity used (labour: actual hours worked) | , |
| `quantity_purchased` | in | quantity purchased (materials; default: the quantity used) (optional) | , |
| `price_variance` | out | price (rate) variance QP (AP - SP); positive = unfavourable | , |
| `quantity_variance` | out | quantity (efficiency) variance SP (AQ - SQ); positive = unfavourable | , |
| `total_variance` | out | price variance + quantity variance | , |
| `price_variance_type` | out | favourable, unfavourable or none | , |
| `quantity_variance_type` | out | favourable, unfavourable or none | , |
| `total_variance_type` | out | favourable, unfavourable or none | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `standard_cost_allowed` | out | standard cost of the actual output SQ x SP | , |

**Applies when:** standard price and standard quantity > 0; actual price, actual quantity and quantity purchased >= 0.

```
$ economics-finance solve direct_cost_variances standard_price=4 standard_quantity=2000
actual_price=4.2 actual_quantity=2100
price_variance = 420
quantity_variance = 400
total_variance = 820
price_variance_type = unfavourable
quantity_variance_type = unfavourable
total_variance_type = unfavourable
standard_cost_allowed = 8000
  by: Standard-cost variances: materials price and quantity, labour rate and efficiency  (PV = QP
(AP - SP), QV = SP (AQ - SQ), total = PV + QV; positive = unfavourable)
  standard_price = 4
  standard_quantity = 2000
  actual_price = 4.2
  actual_quantity = 2100
  assumes: standard price and standard quantity > 0; actual price, actual quantity and quantity
purchased >= 0
```

In the worked example, standard_price = 4, standard_quantity = 2000, actual_price = 4.2, actual_quantity = 2100 go through the equation above and give price_variance = 420, quantity_variance = 400, total_variance = 820, price_variance_type = unfavourable. It applies only when standard price and standard quantity > 0; actual price, actual quantity and quantity purchased >= 0: outside that it says what is wrong instead of returning a number.

## Actuarial mathematics (8)

### Endowment insurance, `endowment_insurance`

Actuarial present value of a benefit paid at the end of the year of death within n years, or at year n if alive then (term insurance + pure endowment).

$$A_{x:\overline{n}|} = A^{1}_{x:\overline{n}|} + v^{n}\,{}_{n}p_{x}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `interest_rate` | in | annual effective interest rate (decimal) | , |
| `term` | in | term n in years | , |
| `benefit` | in | benefit amount (default 1) (optional) | , |
| `apv` | out | B (A^1_{x:n} + nE_x) | , |
| `term_part` | out | B A^1_{x:n} | , |
| `pure_endowment` | out | B v^n n p_x | , |

**Applies when:** 1 <= term <= number of rates.

```
$ economics-finance solve endowment_insurance "mortality_rates=[0.01, 0.02, 0.03]"
interest_rate=0.04 term=3 benefit=1000
apv = 890.4258648
term_part = 53.79672565
pure_endowment = 836.6291392
  by: Endowment insurance  (A_{x:n} = A^1_{x:n} + v^n n p_x)
  interest_rate = 0.04
  benefit = 1000
  assumes: 1 <= term <= number of rates
```

**What the worked example means:** This calculation determines the actuarial present value of an endowment insurance policy. It combines the value of a term insurance benefit, which pays out upon death within the specified term, with a pure endowment benefit, which pays out if the insured survives to the end of the term. The total present value of 890.426 reflects these two components based on the provided mortality rates, interest rate, term length, and benefit amount.

**Life expectancy from a life table, `life_expectancy`**

Curtate expectation of life e_x = sum_{k>=1} k p_x (whole years lived) and the complete expectation e_x + 1/2 (deaths uniform within each year).

$$e_x = \sum_{k=1}^{\infty} {}_k p_x$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `curtate` | out | curtate expectation of life e_x | , |
| `complete` | out | complete expectation e_x + 0.5 (UDD) | , |

**Applies when:** the table ends with q = 1.

```
$ economics-finance solve life_expectancy "mortality_rates=[0.1, 0.3, 0.6, 1]"
curtate = 1.782
complete = 2.282
  by: Life expectancy from a life table  (e_x = sum_{k=1}^{n} k p_x)
  assumes: the table ends with q = 1
```

**What the worked example means:** This calculation determines the average number of future years a person at a specific age is expected to live using the provided mortality rates. The curtate expectation counts only whole years lived and equals 1.782. The complete expectation adjusts this figure by adding half a year to account for deaths occurring uniformly within each year interval, resulting in 2.282. These values serve as fundamental metrics for assessing longevity risk and planning insurance or retirement products based on current survival probabilities.

**Net annual premium for whole life insurance, `net_premium_whole_life`**

The level annual net premium, paid at the start of each year while alive, for a whole life benefit (equivalence principle): P = B A_x / a-due_x.

$$P = B \frac{A_x}{\ddot{a}_x}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `interest_rate` | in | annual effective interest rate (decimal) | , |
| `benefit` | in | benefit amount (default 1) (optional) | , |
| `premium` | out | net annual premium B A_x / a-due_x | , |
| `insurance_apv` | out | B A_x | , |
| `annuity_due` | out | a-due_x | , |

**Applies when:** the table ends with q = 1; 0 < interest_rate < 1.

```
$ economics-finance solve net_premium_whole_life "mortality_rates=[0.1, 0.3, 0.6, 1]"
interest_rate=0.05 benefit=1000
premium = 330.2729832
insurance_apv = 873.9876903
annuity_due = 2.646258503
  by: Net annual premium for whole life insurance  (equivalence principle: P a-due_x = B A_x)
  interest_rate = 0.05
  benefit = 1000
  assumes: the table ends with q = 1; 0 < interest_rate < 1
```

**What the worked example means:** This calculation finds the level annual net premium for whole life insurance, which is 330.273. It applies the equivalence principle by balancing the present value of the death benefit against the present value of premiums paid while alive. The result uses an annual effective interest rate of 0.05 and mortality rates starting at age [0.1, 0.3, 0.6, 1]. The computed present value of the benefit is 873.988, and the present value of the annuity due is 2.64626.

### Survival and death probabilities from a life table, `survival_probability`

t p_x (survive t years), t q_x (die within t years) and t|u q_x (survive t years, then die within the next u years) from one-year mortality rates.

$$ {}_tp_x = \prod_{k=0}^{t-1}(1 - q_{x+k}), \quad {}_{t|u}q_x = {}_tp_x - {}_{t+u}p_x $$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `years` | in | number of years t | , |
| `deferred_years` | in | u, the length of the death window after t (default 1) (optional) | , |
| `survival` | out | t p_x = prod (1 - q) | , |
| `death_probability` | out | t q_x = 1 - t p_x | , |
| `deferred_death_probability` | out | t\|u q_x = t p_x - (t+u) p_x | , |

**Applies when:** 0 <= q <= 1; t + u <= number of rates.

```
$ economics-finance solve survival_probability "mortality_rates=[0.01, 0.02, 0.03, 0.05]" years=3
survival = 0.941094
death_probability = 0.058906
deferred_death_probability = 0.0470547
  by: Survival and death probabilities from a life table  (t p_x = prod_{k<t} (1 - q_{x+k}))
  assumes: 0 <= q <= 1; t + u <= number of rates
```

**What the worked example means:** This calculation determines the likelihood of surviving a specific duration based on annual mortality rates. Using the provided one-year death probabilities, the model computes the cumulative survival chance for 3 years as 0.941094. Consequently, the probability of dying within this period is derived as 0.058906. If a deferred window is specified, the probability of surviving the initial term and then dying in the subsequent interval is calculated as 0.0470547. These values reflect the aggregate risk profile defined by the input mortality data.

**Temporary life annuity-due, `temporary_annuity_due`**

Actuarial present value of up to n annual payments at the start of each year while alive.

$$\ddot{a}_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^k \, {}_kp_x$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `interest_rate` | in | annual effective interest rate (decimal) | , |
| `term` | in | term n in years | , |
| `payment` | in | annual payment (default 1) (optional) | , |
| `apv` | out | P sum_{k<n} v^k k p_x | , |

**Applies when:** 1 <= term <= number of rates.

```
$ economics-finance solve temporary_annuity_due "mortality_rates=[0.01, 0.02, 0.03]"
interest_rate=0.04 term=3 payment=1000
apv = 2848.927515
  by: Temporary life annuity-due  (a-due_{x:n} = sum_{k=0}^{n-1} v^k k p_x)
  interest_rate = 0.04
  payment = 1000
  assumes: 1 <= term <= number of rates
```

**What the worked example means:** This calculation determines the current value of a series of payments made at the start of each year while an individual remains alive. Using an annual payment of 1000, a term of 3 years, an interest rate of 0.04, and the provided mortality rates starting from age x, it computes the expected discounted sum of survival probabilities. The resulting actuarial present value is 2848.93, representing the fair price today for this limited income protection.

**Term life insurance (net single premium), `term_insurance`**

Actuarial present value of a benefit paid at the end of the year of death if death occurs within n years.

$$A^1_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^{k+1} \, {}_kp_x \, q_{x+k}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `interest_rate` | in | annual effective interest rate (decimal) | , |
| `term` | in | term n in years | , |
| `benefit` | in | benefit amount (default 1) (optional) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `apv` | out | B A^1_{x:n} = B sum_{k<n} v^(k+1) k p_x q_x+k | , |

**Applies when:** 1 <= term <= number of rates.

```
$ economics-finance solve term_insurance "mortality_rates=[0.01, 0.02, 0.03]" interest_rate=0.04
term=3 benefit=100000
apv = 5379.672565
  by: Term life insurance (net single premium)  (A^1_{x:n} = sum_{k=0}^{n-1} v^(k+1) k p_x
q_(x+k))
  interest_rate = 0.04
  benefit = 100000
  assumes: 1 <= term <= number of rates
```

**What the worked example means:** This calculation determines the single net premium for a term life insurance policy by summing the discounted probabilities of death occurring in each year of the coverage period. Using an annual effective interest rate of 0.04 and mortality rates starting from age x, it computes the expected present value of a benefit amount of 100000 payable at the end of the year of death if death occurs within the term of 3 years. The resulting actuarial present value is 5379.67, representing the theoretical cost to the insurer for this temporary protection.

### Whole life annuity-due, `life_annuity_due`

Actuarial present value of a payment at the start of every year while the person is alive.

$$\ddot{a}_x = \sum_{k=0}^{\infty} v^k \, {}_kp_x$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `interest_rate` | in | annual effective interest rate (decimal) | , |
| `payment` | in | annual payment (default 1) (optional) | , |
| `apv` | out | P a-due_x = P sum v^k k p_x | , |

**Applies when:** the table ends with q = 1; -1 < interest_rate < 1.

```
$ economics-finance solve life_annuity_due "mortality_rates=[0.1, 0.3, 0.6, 1]"
interest_rate=0.05 payment=1000
apv = 2646.258503
  by: Whole life annuity-due  (a-due_x = sum_k v^k k p_x)
  interest_rate = 0.05
  payment = 1000
  assumes: the table ends with q = 1; -1 < interest_rate < 1
```

**What the worked example means:** This calculation determines the lump-sum value today of a series of annual payments made at the beginning of each year for as long as the individual survives. Using the provided mortality rates and interest rate, the model sums the discounted expected cash flows to arrive at an actuarial present value of 2646.26 for a payment amount of 1000. This figure represents the fair price or reserve required to fund such a lifetime income stream under the specified assumptions.

### Whole life insurance (net single premium), `whole_life_insurance`

Actuarial present value A_x of a benefit paid at the end of the year of death, whenever it occurs.

$$A_x = \sum_{k=0}^{\infty} v^{k+1}\ {}_kp_x\ q_{x+k}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mortality_rates` | in | one-year death probabilities q_x, q_x+1, ... from the current age | , |
| `interest_rate` | in | annual effective interest rate (decimal) | , |
| `benefit` | in | benefit amount (default 1) (optional) | , |
| `apv` | out | B A_x = B sum v^(k+1) k p_x q_x+k | , |

**Applies when:** the table ends with q = 1; -1 < interest_rate < 1.

```
$ economics-finance solve whole_life_insurance "mortality_rates=[0.1, 0.3, 0.6, 1]"
interest_rate=0.05 benefit=1000
apv = 873.9876903
  by: Whole life insurance (net single premium)  (A_x = sum_k v^(k+1) k p_x q_(x+k))
  interest_rate = 0.05
  benefit = 1000
  assumes: the table ends with q = 1; -1 < interest_rate < 1
```

**What the worked example means:** This calculation determines the actuarial present value, or net single premium, required to fund a whole life insurance policy. It sums the discounted probabilities of death for each future year, multiplied by the benefit amount. The result, 873.988, represents the lump sum needed today to cover the expected payout of 1000 at the end of the year of death, given an interest rate of 0.05.

## Bonds (7)

### Bond price change from duration and convexity, `bond_price_change`

The price change of a bond for a change in its yield: duration estimate, duration-plus-convexity estimate and the exact change by repricing.

$$\Delta P \approx -D_{mod}\, P\, \Delta y + \tfrac{1}{2}\, C\, P\, (\Delta y)^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `face` | in | face (par) value repaid at maturity | , |
| `coupon_rate` | in | annual coupon rate (decimal) | , |
| `years` | in | years to maturity (a whole number of coupon periods) | , |
| `ytm` | in | current yield to maturity, nominal annual | , |
| `yield_change` | in | change in the yield (decimal; +1% = 0.01, 25 basis points = 0.0025) | , |
| `frequency` | in | coupons per year (2 = semiannual, default) (optional) | , |
| `duration_estimate` | out | -D_mod P dy | , |
| `duration_convexity_estimate` | out | -D_mod P dy + C P dy^2 / 2 | , |
| `actual_change` | out | exact change by repricing | , |
| `new_price` | out | price at the new yield | , |
| `price` | out | price now | , |

**Applies when:** face > 0; 0 <= coupon_rate < 1; the old and new yields between -100% and 100%; |yield_change| < 50%.

```
$ economics-finance solve bond_price_change face=1000 coupon_rate=0.05 years=10 ytm=0.06
yield_change=0.01 frequency=2
duration_estimate = -70.94863318
duration_convexity_estimate = -67.62635965
actual_change = -67.73665872
new_price = 857.875967
price = 925.6126257
  by: Bond price change from duration and convexity  (dP ~ -D_mod P dy + (1/2) C P dy^2; exact
change by repricing at y + dy)
  face = 1000
  coupon_rate = 0.05
  years = 10
  ytm = 0.06
  yield_change = 0.01
  assumes: face > 0; 0 <= coupon_rate < 1; the old and new yields between -100% and 100%;
|yield_change| < 50%
```

**What the worked example means:** This calculation determines how a bond's price reacts to a shift in yield using duration and convexity. Starting with a current price of 925.613, it computes a linear estimate of -70.9486 and an adjusted estimate of -67.6264. The exact change is found by repricing the bond at the new yield, resulting in a final price of 857.876. This comparison shows the accuracy of the approximation for the given yield change of 0.01.

**Bond price from its yield, `bond_price`**

Price of a fixed-coupon bond on a coupon date from its yield to maturity, with the current yield.

$$P = \sum_{k=1}^{N} \frac{C}{(1 + y/m)^k} + \frac{F}{(1 + y/m)^N}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `face` | in | face (par) value repaid at maturity | , |
| `coupon_rate` | in | annual coupon rate (decimal) | , |
| `years` | in | years to maturity (a whole number of coupon periods) | , |
| `ytm` | in | yield to maturity, nominal annual, compounded at the coupon frequency | , |
| `frequency` | in | coupons per year (2 = semiannual, default) (optional) | , |
| `price` | out | bond price | , |
| `current_yield` | out | annual coupon / price | , |
| `coupon` | out | coupon per period | , |

**Applies when:** face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number, years at most 1000.

```
$ economics-finance solve bond_price face=1000 coupon_rate=0.05 years=10 ytm=0.06 frequency=2
price = 925.6126257
current_yield = 0.05401827785
coupon = 25
  by: Bond price from its yield  (P = C (1 - (1 + y/m)^-N)/(y/m) + F (1 + y/m)^-N, C = F c / m, N
= years x m)
  face = 1000
  coupon_rate = 0.05
  years = 10
  ytm = 0.06
  assumes: face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number, years
at most 1000
```

**What the worked example means:** This calculation determines the fair market price of a fixed-coupon bond, which is 925.613, based on its face value of 1000, annual coupon rate of 0.05, and years to maturity of 10. The price reflects the present value of all future cash flows discounted at the yield to maturity of 0.06. Additionally, the periodic coupon payment is calculated as 25, leading to a current yield of 0.0540183, which represents the annual income relative to the bond's market price.

**Convexity of a bond, `bond_convexity`**

The convexity (1/P) d^2P/dy^2 of a fixed-coupon bond, in years squared.

$$C = \frac{1}{P\,m^2} \sum_{k=1}^{N} \frac{k(k+1)\,CF_k}{(1+y/m)^{k+2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `face` | in | face (par) value repaid at maturity | , |
| `coupon_rate` | in | annual coupon rate (decimal) | , |
| `years` | in | years to maturity (a whole number of coupon periods) | , |
| `ytm` | in | yield to maturity, nominal annual, compounded at the coupon frequency | , |
| `frequency` | in | coupons per year (2 = semiannual, default) (optional) | , |
| `convexity` | out | convexity (1/P) d2P/dy2, years^2 | , |
| `price` | out | bond price | , |

**Applies when:** face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number, years at most 1000.

```
$ economics-finance solve bond_convexity face=1000 coupon_rate=0.05 years=10 ytm=0.06 frequency=2
convexity = 71.78539801
price = 925.6126257
  by: Convexity of a bond  (C = sum CF_k k (k + 1) (1 + y/m)^-(k+2) / (m^2 P))
  face = 1000
  coupon_rate = 0.05
  years = 10
  ytm = 0.06
  assumes: face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number, years
at most 1000
```

**What the worked example means:** This calculation determines the convexity of a fixed-coupon bond, which measures the curvature of its price-yield relationship. Using the face value, coupon rate, time to maturity, yield to maturity, and payment frequency, the model computes the second derivative of price with respect to yield. The

resulting convexity value of 71.7854 indicates how much the duration estimate will deviate for large interest rate changes, while the computed price of 925.613 reflects the present value of all future cash flows.

**Implied forward rate from spot rates, `forward_rate`**

The annually compounded forward rate between t1 and t2 implied by the spot rates for t1 and t2.

$$f_{t_1,t_2} = \left[\frac{(1+s_2)^{t_2}}{(1+s_1)^{t_1}}\right]^{1/(t_2-t_1)} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `spot_rate_1` | in | spot rate to maturity t1 (annual compounding) | , |
| `maturity_1` | in | first maturity t1 in years | , |
| `spot_rate_2` | in | spot rate to maturity t2 (annual compounding) | , |
| `maturity_2` | in | second maturity t2 > t1 in years | , |
| `forward_rate` | out | forward rate f(t1, t2), annual compounding | , |

**Applies when:** -1 < spot rates < 1; 0 <= t1 < t2.

```
$ economics-finance solve forward_rate spot_rate_1=0.04 maturity_1=1 spot_rate_2=0.05
maturity_2=2
forward_rate = 0.06009615385
  by: Implied forward rate from spot rates  ((1 + s2)^t2 = (1 + s1)^t1 (1 + f)^(t2 - t1))
  spot_rate_1 = 0.04
  maturity_1 = 1
  spot_rate_2 = 0.05
  maturity_2 = 2
  assumes: -1 < spot rates < 1; 0 <= t1 < t2
```

**What the worked example means:** The annually compounded forward rate between maturity 1 and maturity 2 is calculated using the spot rates of 0.04 for the first period and 0.05 for the second. The resulting forward rate is 0.0600962, which represents the break-even interest rate that prevents arbitrage between investing directly to the longer date versus rolling over a shorter investment.

**Macaulay and modified duration of a bond, `bond_duration`**

The Macaulay duration (PV-weighted average time to the cash flows, years) and the modified duration (the price's sensitivity -dP/dy / P) of a fixed-coupon bond.

$$D_{mac} = \frac{1}{P}\sum_{k=1}^{N}\frac{k}{m}\,\frac{CF_k}{(1+y/m)^k}, \quad D_{mod} = \frac{D_{mac}}{1+y/m}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `face` | in | face (par) value repaid at maturity | , |
| `coupon_rate` | in | annual coupon rate (decimal) | , |
| `years` | in | years to maturity (a whole number of coupon periods) | , |
| `ytm` | in | yield to maturity, nominal annual, compounded at the coupon frequency | , |
| `frequency` | in | coupons per year (2 = semiannual, default) (optional) | , |
| `macaulay_duration` | out | Macaulay duration in years | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `modified_duration` | out | modified duration = Macaulay / (1 + y/m) | , |
| `price` | out | bond price | , |

**Applies when:** face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number, years at most 1000.

```
$ economics-finance solve bond_duration face=1000 coupon_rate=0.05 years=10 ytm=0.06 frequency=2
macaulay_duration = 7.89499734
modified_duration = 7.665045961
price = 925.6126257
  by: Macaulay and modified duration of a bond  (D_mac = sum (k/m) PV_k / P; D_mod = D_mac / (1 +
y/m))
  face = 1000
  coupon_rate = 0.05
  years = 10
  ytm = 0.06
  assumes: face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number, years
at most 1000
```

**What the worked example means:** This calculation determines the time-weighted average maturity of cash flows, known as Macaulay duration, and its interest rate sensitivity measure, called modified duration. Using a face value of 1000, an annual coupon rate of 0.05, and a yield to maturity of 0.06 over 10 years with 2 payments per year, the resulting bond price is 925.613. The computed Macaulay duration of 7.895 represents the weighted average time until cash flows are received, while the modified duration of 7.66505 estimates the percentage change in the bond's price for a one-unit shift in yield.

### Price of a zero-coupon bond, `zero_coupon_bond`

Price of a bond that pays only its face value at maturity, at a yield with a stated compounding.

$$P = \frac{F}{(1 + y/m)^{mt}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `face` | in | face value paid at maturity | , |
| `ytm` | in | yield (decimal), with the stated compounding | , |
| `years` | in | years to maturity | , |
| `compounding` | in | annual (default), semiannual, quarterly, monthly or continuous (optional) | , |
| `price` | out | price F x discount factor | , |
| `discount_factor` | out | (1 + y/m)^(-m t) or e^(-y t) | , |

**Applies when:** face > 0; -1 < ytm < 1; years >= 0.

```
$ economics-finance solve zero_coupon_bond face=1000 ytm=0.05 years=10 compounding=semiannual
price = 610.2709429
discount_factor = 0.6102709429
  by: Price of a zero-coupon bond  (P = F (1 + y/m)^(-m t), or F e^(-y t) when continuous)
  face = 1000
  ytm = 0.05
  years = 10
  compounding = semiannual
  assumes: face > 0; -1 < ytm < 1; years >= 0
```

**What the worked example means:** This calculation determines the present value of a zero-coupon bond, which pays only its face value at maturity. Using an input face value of 1000, a yield to maturity of 0.05, and a time horizon of 10, the model applies the specified compounding frequency to discount future cash flows back to today. The resulting market price is 610.271, reflecting a discount factor of 0.610271. This value represents what an investor would pay now to receive the full principal amount at the end of the term, assuming no interim interest payments.

**Yield to maturity of a bond, `bond_ytm`**

The yield to maturity (nominal annual, compounded at the coupon frequency) that equates the bond's price to the present value of its coupons and face value.

$$\text{price} = \sum_{k=1}^{N} \frac{C}{(1+y/m)^k} + \frac{F}{(1+y/m)^N}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price` | in | bond price (clean = dirty on a coupon date) | , |
| `face` | in | face (par) value repaid at maturity | , |
| `coupon_rate` | in | annual coupon rate (decimal) | , |
| `years` | in | years to maturity (a whole number of coupon periods) | , |
| `frequency` | in | coupons per year (2 = semiannual, default) (optional) | , |
| `ytm` | out | yield to maturity, nominal annual | , |
| `current_yield` | out | annual coupon / price | , |

**Applies when:** price > 0; face > 0; 0 <= coupon_rate < 1; years x frequency a whole number, years at most 1000; a yield between -50% and 100% per period.

```
$ economics-finance solve bond_ytm price=925.61 face=1000 coupon_rate=0.05 years=10 frequency=2
ytm = 0.06000037008
current_yield = 0.05401843109
  by: Yield to maturity of a bond  (bisection on P(y) = price (P falls as y rises, so the yield
is unique))
  price = 925.61
  face = 1000
  coupon_rate = 0.05
  years = 10
  assumes: price > 0; face > 0; 0 <= coupon_rate < 1; years x frequency a whole number, years at
most 1000; a yield between -50% and 100% per period
```

**What the worked example means:** This calculation determines the yield to maturity, which is the internal rate of return earned by holding a bond until it matures. It solves for the discount rate that equates the present value of all future coupon payments and the face value repayment to the bond's current market price. The result shows that for a

bond priced at 925.61 with a face value of 1000, paying an annual coupon rate of 0.05 over 10 years with a frequency of 2, the nominal annual yield to maturity is 0.0600004. Additionally, the current yield, representing the annual income relative to the price, is calculated as 0.0540184.

## Capital budgeting (8)

### Discounted payback period, `discounted_payback`

Time until the cumulative DISCOUNTED cash flow turns non-negative (linear within the period).

$$\sum_{s \le t} \frac{CF_s}{(1+r)^s} \ge 0$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | discount rate per period (decimal) | , |
| `cash_flows` | in | cash flows [CF0 < 0, CF1, ..., CFn > 0] | , |
| `discounted_payback` | out | discounted payback period in periods | , |

**Applies when:** CF0 < 0; every later cash flow > 0; recovered within the flows; -1 < rate < 1.

```
$ economics-finance solve discounted_payback rate=0.1 "cash_flows=[-1000, 300, 400, 500, 200]"
discounted_payback = 3.154
  by: Discounted payback period  (payback of the discounted flows CFt / (1 + r)^t)
  rate = 0.1
  assumes: CF0 < 0; every later cash flow > 0; recovered within the flows; -1 < rate < 1
```

**What the worked example means:** This calculation determines the time required for an investment to recover its initial cost using cash flows adjusted for the time value of money. Given a discount rate of 0.1, the cumulative discounted cash flow becomes non-negative after 3.154 periods. This metric provides a more realistic assessment of liquidity and risk than simple payback by accounting for the fact that future money is worth less than present money.

### Equivalent annual annuity, `equivalent_annual_annuity`

The level payment over the project's life with the same NPV (for comparing projects of unequal lives).

$$EAA = \frac{NPV\, r}{1-(1+r)^{-n}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | discount rate per period (decimal) | , |
| `cash_flows` | in | cash flows [CF0, CF1, ..., CFn] | , |
| `eaa` | out | equivalent annual annuity NPV r / (1 - (1 + r)^-n) | , |
| `npv` | out | net present value | , |

**Applies when:** -1 < rate < 1; at least two cash flows.

```
$ economics-finance solve equivalent_annual_annuity rate=0.1 "cash_flows=[-1000, 300, 400, 500]"
eaa = -8.459214502
npv = -21.03681443
  by: Equivalent annual annuity  (EAA = NPV r / (1 - (1 + r)^-n))
  rate = 0.1
  assumes: -1 < rate < 1; at least two cash flows
```

**What the worked example means:** This calculation determines the equivalent annual annuity, which represents the constant yearly cash flow that has the same net present value as the original project's uneven cash flows. By using the discount rate of 0.1, the method converts the total net present value of -21.0368 into an annualized figure of -8.45921. This allows for a direct comparison between projects that have different lifespans or unequal durations.

**Internal rate of return, `irr`**

The discount rate at which the NPV of [CF0, ..., CFn] is zero; given only when it is unique (the cash flows change sign exactly once).

$$\sum_{t=0}^{n} \frac{CF_t}{(1+\mathrm{IRR})^t} = 0$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `cash_flows` | in | cash flows [CF0, CF1, ..., CFn], CF0 at time 0 | , |
| `irr` | out | internal rate of return per period | , |

**Applies when:** the cash flows change sign exactly once (otherwise the IRR is not unique and none is given).

```
$ economics-finance solve irr "cash_flows=[-1000, 300, 400, 500]"
irr = 0.08896339469
  by: Internal rate of return  (bisection on v = 1/(1 + r) for the unique positive root of sum
CFt v^t, Newton polishing)
  assumes: the cash flows change sign exactly once (otherwise the IRR is not unique and none is
given)
```

**What the worked example means:** This calculation determines the internal rate of return, which is the specific discount rate where the sum of the discounted cash flows equals zero. Using the provided sequence of cash flows starting with [-1000, 300, 400, 500], the algorithm finds this break-even interest rate. The resulting value, 0.0889634, represents the project's implied annualized yield. This metric allows for direct comparison against hurdle rates or other investment opportunities by standardizing returns into a single percentage figure, assuming the cash flow pattern changes sign exactly once to ensure uniqueness.

**Modified internal rate of return, `mirr`**

MIRR: outflows discounted at the finance rate, inflows compounded to the end at the reinvestment rate.

$$\mathrm{MIRR} = \left(\frac{FV_+}{PV_-}\right)^{1/n} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `cash_flows` | in | cash flows [CF0, ..., CFn], CF0 at time 0 | , |
| `finance_rate` | in | finance rate for outflows (decimal) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `reinvest_rate` | in | reinvestment rate for inflows (decimal) | , |
| `mirr` | out | modified IRR per period | , |
| `terminal_value` | out | inflows compounded to period n | , |
| `pv_outflows` | out | present value of the outflows | , |

**Applies when:** at least one inflow and one outflow; -1 < rates < 1.

```
$ economics-finance solve mirr "cash_flows=[-1000, 300, 400, 500]" finance_rate=0.1
reinvest_rate=0.12
mirr = 0.09815669245
terminal_value = 1324.32
pv_outflows = 1000
  by: Modified internal rate of return  (MIRR = (FV of inflows at the reinvestment rate / PV of
outflows at the finance rate)^(1/n) - 1)
  finance_rate = 0.1
  reinvest_rate = 0.12
  assumes: at least one inflow and one outflow; -1 < rates < 1
```

**What the worked example means:** This calculation determines the modified internal rate of return by first compounding all positive cash inflows to the end of the project at the reinvestment rate, resulting in a terminal value of 1324.32. Simultaneously, all negative cash outflows are discounted back to time zero using the finance rate, yielding a present value of 1000. The final metric, 0.0981567, represents the single discount rate that equates these two adjusted values over the project's life, offering a more realistic profitability measure than standard methods.

### Net present value, `npv`

NPV of cash flows [CF0, CF1, ..., CFn] (CF0 now, CFt at the end of period t) at a discount rate per period.

$$NPV = \sum_{t=0}^{n} \frac{CF_t}{(1+r)^t}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | discount rate per period (decimal) | , |
| `cash_flows` | in | cash flows [CF0, CF1, ..., CFn], CF0 at time 0 | , |
| `npv` | out | net present value | , |

**Applies when:** -1 < rate < 1; at least two cash flows.

```
$ economics-finance solve npv rate=0.1 "cash_flows=[-1000, 300, 400, 500]"
npv = -21.03681443
  by: Net present value  (NPV = sum CFt / (1 + r)^t, t = 0..n)
  rate = 0.1
  assumes: -1 < rate < 1; at least two cash flows
```

**What the worked example means:** This calculation determines the net present value by summing each cash flow divided by one plus the discount rate raised to the power of its time period. The resulting value of -21.0368 represents the current worth of the future cash flows given a discount rate of 0.1. A positive outcome suggests the investment generates returns above the required threshold, while a negative result indicates it falls short.

### NPV crossover rate of two projects, `crossover_rate`

The discount rate at which two projects have equal NPV (the IRR of the difference A - B); given only when it is unique.

$$\sum_t \frac{CF_t^A - CF_t^B}{(1+r^*)^t} = 0$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `cash_flows_a` | in | cash flows of project A [CF0, ..., CFn] | , |
| `cash_flows_b` | in | cash flows of project B, same periods | , |
| `crossover_rate` | out | rate at which NPV(A) = NPV(B) | , |
| `npv_at_crossover` | out | the common NPV at that rate | , |

**Applies when:** equal lengths; the difference A - B changes sign exactly once.

```
$ economics-finance solve crossover_rate "cash_flows_a=[-1000, 100, 1200]" "cash_flows_b=[-1000,
700, 500]"
crossover_rate = 0.1666666667
npv_at_crossover = -32.65306122
  by: NPV crossover rate of two projects  (IRR of the incremental cash flows A - B)
  assumes: equal lengths; the difference A - B changes sign exactly once
```

**What the worked example means:** This calculation determines the specific discount rate at which the net present value of project A equals that of project B, effectively finding the internal rate of return for the incremental cash flows defined by subtracting the cash flows of project B from those of project A. The result, 0.166667, represents the threshold cost of capital where the preference between these two mutually exclusive projects switches. At this precise rate, both projects yield an identical net present value of -32.6531, meaning neither is financially superior to the other based solely on this metric. This analysis relies on the assumption that the projects have equal lifespans and that the difference in their cash flow streams changes sign exactly once, ensuring a unique solution.

### Payback period, `payback_period`

Time until the cumulative cash flow turns non-negative, interpolating linearly within the period (cash arriving evenly during the period).

$$\text{payback} = (t-1) + \frac{-\sum_{s<t} CF_s}{CF_t}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `cash_flows` | in | cash flows [CF0 < 0, CF1, ..., CFn > 0] | , |
| `payback` | out | payback period in periods | , |

**Applies when:** CF0 < 0; every later cash flow > 0; the investment is recovered within the flows.

```
$ economics-finance solve payback_period "cash_flows=[-1000, 300, 400, 500]"
payback = 2.6
  by: Payback period  (first t with cumulative flow >= 0; payback = (t - 1) + (unrecovered at t -
1) / CFt)
  assumes: CF0 < 0; every later cash flow > 0; the investment is recovered within the flows
```

**What the worked example means:** This calculation determines the time required for cumulative cash inflows to recover the initial investment, resulting in a payback period of 2.6 periods. It assumes that cash flows occur evenly

throughout each period and that all subsequent cash flows are positive. The result indicates how quickly the capital invested in the project is returned, serving as a simple measure of liquidity risk rather than overall profitability.

**Profitability index, `profitability_index`**

PV of the future cash flows divided by the initial investment (benefit-cost ratio).

$$PI = \frac{\sum_{t=1}^{n} CF_t (1+r)^{-t}}{-CF_0}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | discount rate per period (decimal) | , |
| `cash_flows` | in | cash flows [CF0 < 0, CF1, ..., CFn] | , |
| `profitability_index` | out | PV(CF1..CFn) / \|CF0\| | , |
| `pv_future` | out | PV of CF1..CFn | , |
| `npv` | out | net present value | , |

**Applies when:** CF0 < 0; -1 < rate < 1.

```
$ economics-finance solve profitability_index rate=0.1 "cash_flows=[-1000, 300, 400, 500]"
profitability_index = 0.9789631856
pv_future = 978.9631856
npv = -21.03681443
  by: Profitability index  (PI = sum_{t>=1} CFt/(1+r)^t / (-CF0))
  rate = 0.1
  assumes: CF0 < 0; -1 < rate < 1
```

**What the worked example means:** This calculation determines the profitability index, which represents the ratio of the present value of future cash flows to the absolute value of the initial outlay. Discounting each subsequent cash flow at the rate of 0.1 and summing them to reach 978.963 compares this benefit against the cost embedded in [-1000, 300, 400, 500]. The resulting index of 0.978963 indicates how much value is generated for every unit of currency invested, serving as a key metric for assessing project efficiency.

## Consumer finance (2)

**Credit card finance charge (average daily balance), `credit_card_finance_charge`**

Average daily balance of a credit card billing cycle and the finance charge on it, from the opening balance and the dated purchases and payments of the cycle.

$$\overline{B} = \frac{1}{D}\sum_{d=1}^{D} B_d, \qquad FC = \overline{B} \cdot \frac{\mathrm{APR}\, D}{365} \text{ or } \overline{B} \cdot \frac{\mathrm{APR}}{12}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `opening_balance` | in | balance owed at the start of the cycle (previous balance) | , |
| `apr` | in | annual percentage rate of the card (decimal; 18% = 0.18) | , |
| `billing_days` | in | number of days in the billing cycle | , |
| `rate_basis` | in | 'daily' (APR/365 for each day of the cycle) or 'monthly' (APR/12) | , |
| `transaction_days` | in | day of the cycle (1 = first day) of each purchase or payment (optional) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `transaction_amounts` | in | amount of each transaction: purchases positive, payments negative (optional) | , |
| `average_daily_balance` | out | mean of the end-of-day balances over the cycle | , |
| `finance_charge` | out | interest charged for the cycle: ADB x cycle rate | , |
| `cycle_rate` | out | rate applied for the cycle (APR x days / 365, or APR / 12) | , |
| `closing_balance` | out | balance at the end of the cycle before the finance charge | , |
| `new_balance` | out | closing balance plus the finance charge | , |

**Applies when:** opening_balance >= 0; 0 <= apr < 1; billing_days a whole number 1..366; rate_basis daily or monthly; transaction days whole numbers 1..billing_days, one per amount; the balance never becomes negative.

```
$ economics-finance solve credit_card_finance_charge opening_balance=1500 apr=0.18
billing_days=30 rate_basis=daily "transaction_days=[11, 21]" "transaction_amounts=[-500, 200]"
average_daily_balance = 1233.333333
finance_charge = 18.24657534
cycle_rate = 0.01479452055
closing_balance = 1200
new_balance = 1218.246575
  by: Credit card finance charge (average daily balance)  (ADB = (D B0 + sum a_k (D - d_k + 1)) /
D; finance charge = ADB x APR x D / 365 (daily) or ADB x APR / 12 (monthly))
  opening_balance = 1500
  apr = 0.18
  rate_basis = daily
  assumes: opening_balance >= 0; 0 <= apr < 1; billing_days a whole number 1..366; rate_basis
daily or monthly; transaction days whole numbers 1..billing_days, one per amount; the balance
never becomes negative
```

In the worked example, opening_balance = 1500, apr = 0.18, billing_days = 30, rate_basis = daily, transaction_days = [11, 21], transaction_amounts = [-500, 200] go through the equation above and give average_daily_balance = 1233.333333, finance_charge = 18.24657534, cycle_rate = 0.01479452055, closing_balance = 1200. It applies only when opening_balance >= 0; 0 <= apr < 1; billing_days a whole number 1..366; rate_basis daily or monthly; transaction days whole numbers 1..billing_days, one per amount; the balance never becomes negative: outside that it says what is wrong instead of returning a number.

### Income tax from progressive brackets, `income_tax`

Tax owed on a taxable income under a progressive bracket table, with the marginal and the effective (average) tax rate.

$$T(x) = \sum_k r_k \left(\min(x, t_{k+1}) - t_k\right)^+, \qquad \text{effective rate} = \frac{T(x)}{x}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `taxable_income` | in | taxable income (after deductions) | , |
| `bracket_thresholds` | in | lower bound of each bracket, starting at 0, increasing | , |
| `bracket_rates` | in | tax rate of each bracket (decimals or percentages) | , |
| `tax` | out | tax owed: each bracket's rate on the income inside it | , |
| `marginal_rate` | out | rate on the next unit of income | , |
| `effective_rate` | out | tax / taxable income | , |
| `after_tax_income` | out | taxable income minus the tax | , |

**Applies when:** taxable_income > 0; first threshold 0, thresholds strictly increasing; one rate per threshold, 0 <= rate < 1.

```
$ economics-finance solve income_tax taxable_income=50000 "bracket_thresholds=[0, 10000, 40000]"
"bracket_rates=[0.1, 0.2, 0.3]"
tax = 10000
marginal_rate = 0.3
effective_rate = 0.2
after_tax_income = 40000
  by: Income tax from progressive brackets  (tax = sum_k r_k (min(x, t_{k+1}) - t_k)^+; marginal
= r_j with t_j <= x < t_{j+1}; effective = tax / x)
  taxable_income = 50000
  assumes: taxable_income > 0; first threshold 0, thresholds strictly increasing; one rate per
threshold, 0 <= rate < 1
```

In the worked example, taxable_income = 50000, bracket_thresholds = [0, 10000, 40000], bracket_rates = [0.1, 0.2, 0.3] go through the equation above and give tax = 10000, marginal_rate = 0.3, effective_rate = 0.2, after_tax_income = 40000. It applies only when taxable_income > 0; first threshold 0, thresholds strictly increasing; one rate per threshold, 0 <= rate < 1: outside that it says what is wrong instead of returning a number.

## Econometrics (5)

### Constant elasticity from a log-log regression, `log_log_elasticity`

The price elasticity estimated as the slope of ln Q on ln P (constant-elasticity demand Q = A P^e).

$$\ln Q = \ln A + \varepsilon \ln P$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `prices` | in | observed prices | , |
| `quantities` | in | quantities at those prices | , |
| `elasticity` | out | slope of ln Q on ln P | , |
| `intercept` | out | ln A | , |
| `r_squared` | out | R^2 of the log-log fit | , |

**Applies when:** positive values; prices not all equal.

```
$ economics-finance solve log_log_elasticity "prices=[1, 2, 4, 8]" "quantities=[100, 50, 25,
12.5]"
elasticity = -1
intercept = 4.605170186
r_squared = 1
  by: Constant elasticity from a log-log regression  (OLS of ln Q on ln P)
  assumes: positive values; prices not all equal
```

**What the worked example means:** This calculation determines the constant price elasticity of demand by performing ordinary least squares regression on the natural logarithm of quantity against the natural logarithm of price. The slope coefficient, reported as -1, represents the percentage change in quantity demanded for a one percent change in price. The intercept, shown as 4.60517, corresponds to the natural log of the scale parameter A, while the goodness of fit is indicated by the R-squared value of 1.

### Correlation and covariance of two series, `correlation`

Pearson correlation and sample covariance (n - 1) of two paired series, such as the returns of two assets.

$$r = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 \sum (y - \bar{y})^2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `x` | in | first series | , |
| `y` | in | second series | , |
| `correlation` | out | Pearson r | , |
| `covariance` | out | sample covariance | , |
| `r_squared` | out | r^2 | , |

**Applies when:** same length, at least 3; neither series constant.

```
$ economics-finance solve correlation "x=[1, 2, 3, 4, 5]" "y=[2, 4, 5, 4, 5]"
correlation = 0.7745966692
covariance = 1.5
r_squared = 0.6
  by: Correlation and covariance of two series  (r = S_xy / sqrt(S_xx S_yy))
  assumes: same length, at least 3; neither series constant
```

**What the worked example means:** This calculation determines the Pearson correlation and sample covariance for two paired series, such as asset returns. The resulting correlation coefficient of 0.774597 measures how strongly the variables move together on a scale from negative one to positive one. A value near zero suggests little linear association, while values approaching one or negative one indicate a strong direct or inverse relationship. The associated sample covariance of 1.5 quantifies the joint variability of the two series in their original units, providing context for the magnitude of their co-movement relative to their individual variances.

### Durbin-Watson statistic, `durbin_watson`

Durbin-Watson statistic of regression residuals in time order (about 2: no first-order autocorrelation; toward 0: positive; toward 4: negative).

$$DW = \frac{\sum_{t=2}^{n} (e_t - e_{t-1})^2}{\sum_{t=1}^{n} e_t^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `residuals` | in | regression residuals in time order | , |
| `durbin_watson` | out | sum (e_t - e_t-1)^2 / sum e_t^2 | , |

**Applies when:** residuals not all zero.

```
$ economics-finance solve durbin_watson "residuals=[0.5, -0.3, 0.2, -0.4, 0.1]"
durbin_watson = 2.727272727
  by: Durbin-Watson statistic  (DW = sum_{t>=2} (e_t - e_{t-1})^2 / sum_t e_t^2)
  assumes: residuals not all zero
```

**What the worked example means:** The Durbin-Watson statistic measures the presence of first-order autocorrelation in the residuals from a regression analysis. It is calculated using the time-ordered residuals provided as input, specifically comparing the sum of squared differences between consecutive residuals to the total sum of

squared residuals. The resulting value of 2.72727 suggests the degree of correlation: values near two indicate no significant autocorrelation, while values approaching zero suggest positive autocorrelation and values approaching four suggest negative autocorrelation.

**Multiple linear regression (OLS), `multiple_regression`**

Least-squares coefficients of y = b0 + b1 x1 + ... + bk xk (intercept added), R^2 and adjusted R^2.

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `X` | in | regressor matrix, one row per observation, one column per variable (no constant column) | , |
| `y` | in | dependent variable, one value per row of X | , |
| `coefficients` | out | [b0 (intercept), b1, ..., bk] | , |
| `r_squared` | out | 1 - SSR/SST | , |
| `adj_r_squared` | out | 1 - (1 - R^2)(n - 1)/(n - k - 1) | , |

**Applies when:** one row of X per value of y; n > k + 1; columns not collinear; y not constant.

```
$ economics-finance solve multiple_regression "X=[[1, 2], [2, 1], [3, 4], [4, 3], [5, 6]]" "y=[5,
5, 10, 10, 15]"
coefficients = [1.250000000000001,1.2499999999999973,1.2500000000000024]
r_squared = 1
adj_r_squared = 1
  by: Multiple linear regression (OLS)  (QR decomposition (modified Gram-Schmidt) of [1, X], back
substitution)
  assumes: one row of X per value of y; n > k + 1; columns not collinear; y not constant
```

**What the worked example means:** This calculation determines the best-fit linear relationship between the dependent variable and multiple independent variables using ordinary least squares. It produces a set of coefficients, where the first value represents the intercept and subsequent values indicate the slope for each predictor, alongside metrics like 1 that quantify how well the model explains the variance in the data.

**Simple linear regression (OLS), `ols_simple`**

Ordinary least squares fit y = a + b x: slope, intercept, R^2 and the standard error of the slope.

$$\hat{\beta} = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}, \quad \hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `x` | in | explanatory variable values | , |
| `y` | in | dependent variable values | , |
| `slope` | out | b = S_xy / S_xx | , |
| `intercept` | out | a = mean y - b mean x | , |
| `r_squared` | out | 1 - SSR/SST | , |
| `slope_std_error` | out | sqrt(SSR/(n - 2) / S_xx) | , |
| `n` | out | observations | , |

**Applies when:** same number of x and y, at least 3; neither variable constant.

```
$ economics-finance solve ols_simple "x=[1, 2, 3, 4, 5]" "y=[2.1, 3.9, 6.2, 7.8, 10.1]"
slope = 1.99
intercept = 0.05
r_squared = 0.9973053289
slope_std_error = 0.05972157622
n = 5
  by: Simple linear regression (OLS)  (least squares normal equations)
  assumes: same number of x and y, at least 3; neither variable constant
```

**What the worked example means:** This calculation determines the best-fit straight line through the data using ordinary least squares. The slope, 1.99, indicates how much the dependent variable changes for each unit increase in the explanatory variable. The intercept, 0.05, shows the expected value of the dependent variable when the explanatory variable is zero. With 5 observations, the model explains 0.997305 of the variation in the dependent variable, while the standard error of the slope is 0.0597216.

## International trade (1)

### Comparative advantage and opportunity cost, `comparative_advantage`

Opportunity costs of two goods in two countries from output per worker (or hours per unit), who has the absolute and the comparative advantage in each good, and the range of terms of trade that benefits both.

$$OC_X^A = \frac{y_A}{x_A}\ \text{(output)} \quad \text{or} \quad \frac{h_{X,A}}{h_{Y,A}}\ \text{(hours)}, \qquad \min(OC_X^A, OC_X^B) \le p_X \le \max(OC_X^A, OC_X^B)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a_good_x` | in | country A, good X: output per worker (basis output) or hours per unit (basis hours) | , |
| `a_good_y` | in | country A, good Y: output per worker or hours per unit | , |
| `b_good_x` | in | country B, good X: output per worker or hours per unit | , |
| `b_good_y` | in | country B, good Y: output per worker or hours per unit | , |
| `basis` | in | 'output' (units made per worker or hour) or 'hours' (hours needed per unit) | , |
| `opportunity_cost_x_a` | out | country A's opportunity cost of X, in units of Y | , |
| `opportunity_cost_y_a` | out | country A's opportunity cost of Y, in units of X | , |
| `opportunity_cost_x_b` | out | country B's opportunity cost of X, in units of Y | , |
| `opportunity_cost_y_b` | out | country B's opportunity cost of Y, in units of X | , |
| `comparative_advantage_x` | out | who should specialise in X: country A, country B or neither | , |
| `comparative_advantage_y` | out | who should specialise in Y: country A, country B or neither | , |
| `absolute_advantage_x` | out | who is more productive in X: country A, country B or equal | , |
| `absolute_advantage_y` | out | who is more productive in Y: country A, country B or equal | , |
| `terms_low` | out | lowest mutually beneficial price of X, in units of Y | , |
| `terms_high` | out | highest mutually beneficial price of X, in units of Y | , |

**Applies when:** all four numbers > 0; basis output or hours.

```
$ economics-finance solve comparative_advantage a_good_x=6 a_good_y=3 b_good_x=1 b_good_y=2
basis=output
opportunity_cost_x_a = 0.5
opportunity_cost_y_a = 2
opportunity_cost_x_b = 2
opportunity_cost_y_b = 0.5
comparative_advantage_x = country A
comparative_advantage_y = country B
absolute_advantage_x = country A
absolute_advantage_y = country A
terms_low = 0.5
terms_high = 2
  by: Comparative advantage and opportunity cost  (output basis: OC of X = (Y per worker) / (X
per worker); hours basis: OC of X = (hours per X) / (hours per Y); the lower OC of X has the
comparative advantage in X; terms of trade between the two OCs)
  a_good_x = 6
  a_good_y = 3
  b_good_x = 1
  b_good_y = 2
  basis = output
  assumes: all four numbers > 0; basis output or hours
```

In the worked example, a_good_x = 6, a_good_y = 3, b_good_x = 1, b_good_y = 2, basis = output go through the equation above and give opportunity_cost_x_a = 0.5, opportunity_cost_y_a = 2, opportunity_cost_x_b = 2, opportunity_cost_y_b = 0.5. It applies only when all four numbers > 0; basis output or hours: outside that it says what is wrong instead of returning a number.

## Loans and annuities (5)

### Cumulative interest of a loan, `loan_cumulative_interest`

Total interest paid and principal repaid over the first k payments of an amortizing loan.

$$\sum_{j=1}^{k} I_j = k\,PMT - (P - B_k)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount borrowed | , |
| `annual_rate` | in | nominal annual interest rate (decimal), compounded at the payment frequency | , |
| `years` | in | term of the loan in years | , |
| `payments_made` | in | number of payments counted (the first k) | , |
| `payments_per_year` | in | payments per year (12 = monthly, default) (optional) | , |
| `cumulative_interest` | out | interest paid in payments 1..k | , |
| `principal_repaid` | out | principal repaid in payments 1..k | , |

**Applies when:** principal > 0; 0 <= annual_rate < 1; 1 <= payments_made <= number of payments.

```
$ economics-finance solve loan_cumulative_interest principal=200000 annual_rate=0.06 years=30
payments_made=12
cumulative_interest = 11933.18918
principal_repaid = 2456.023425
  by: Cumulative interest of a loan  (principal repaid = P - B_k; interest = k PMT - (P - B_k))
  principal = 200000
  annual_rate = 0.06
  years = 30
  assumes: principal > 0; 0 <= annual_rate < 1; 1 <= payments_made <= number of payments
```

**What the worked example means:** Over the first 12 payments, you have paid 11933.2 in interest and repaid 2456.02 of the principal.

**Interest and principal in a loan payment, `loan_payment_split`**

How payment number k of an amortizing loan splits into interest and principal repayment.

$$I_k = i\,B_{k-1}, \quad P_k = PMT - I_k$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount borrowed | , |
| `annual_rate` | in | nominal annual interest rate (decimal), compounded at the payment frequency | , |
| `years` | in | term of the loan in years | , |
| `payment_number` | in | which payment (1 = the first) | , |
| `payments_per_year` | in | payments per year (12 = monthly, default) (optional) | , |
| `interest_part` | out | interest in payment k | , |
| `principal_part` | out | principal repaid by payment k | , |
| `payment` | out | level payment | , |

**Applies when:** principal > 0; 0 <= annual_rate < 1; 1 <= payment_number <= number of payments.

```
$ economics-finance solve loan_payment_split payment_number=12 principal=200000 annual_rate=0.06
years=30
interest_part = 988.7715305
principal_part = 210.3295198
payment = 1199.10105
  by: Interest and principal in a loan payment  (interest = i B_(k-1), principal = PMT -
interest)
  principal = 200000
  annual_rate = 0.06
  years = 30
  assumes: principal > 0; 0 <= annual_rate < 1; 1 <= payment_number <= number of payments
```

**What the worked example means:** This calculation determines how a specific loan payment is split between interest expense and principal repayment. Using the annual rate, term, and frequency, the level payment amount is found to be 1199.1. For the selected period, the outstanding balance generates an interest charge of 988.772, leaving 210.33 to reduce the borrowed principal. This breakdown highlights that early payments are typically weighted more heavily toward interest, while later payments apply more funds directly against the debt.

**Loan payment (amortizing loan), `loan_payment`**

The level payment that repays a loan over its term, with the total paid and the total interest.

$$PMT = \frac{P\,i}{1 - (1+i)^{-n}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount borrowed | , |
| `annual_rate` | in | nominal annual interest rate (decimal), compounded at the payment frequency | , |
| `years` | in | term of the loan in years | , |
| `payments_per_year` | in | payments per year (12 = monthly, default) (optional) | , |
| `payment` | out | level payment per period | , |
| `number_of_payments` | out | number of payments n = years x m | , |
| `total_paid` | out | payment x n | , |
| `total_interest` | out | total paid - principal | , |

**Applies when:** principal > 0; 0 <= annual_rate < 1; years x payments_per_year a whole number, years at most 1000.

```
$ economics-finance solve loan_payment principal=200000 annual_rate=0.06 years=30
payment = 1199.10105
number_of_payments = 360
total_paid = 431676.3781
total_interest = 231676.3781
  by: Loan payment (amortizing loan)  (PMT = P i / (1 - (1 + i)^-n), i = annual rate / m, n =
years x m)
  principal = 200000
  annual_rate = 0.06
  years = 30
  assumes: principal > 0; 0 <= annual_rate < 1; years x payments_per_year a whole number, years
at most 1000
```

**What the worked example means:** This calculation determines the fixed periodic payment required to fully amortize a loan. Using the principal of 200000 and an annual interest rate of 0.06, the model computes that each payment must be 1199.1. This ensures the balance reaches zero after 360 payments, resulting in a total paid amount of 431676 and total interest costs of 231676.

### Number of payments to repay a loan, `number_of_periods`

How many periodic payments repay a present value at a rate per period (a fractional answer means the last payment is smaller).

$$n = \frac{-\ln\left(1 - \frac{PV\,r}{PMT}\right)}{\ln(1+r)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `present_value` | in | amount borrowed (present value) | , |
| `payment` | in | payment per period | , |
| `rate` | in | interest rate per period (decimal) | , |
| `timing` | in | 'ordinary' (end of each period, default) or 'due' (beginning) (optional) | , |
| `periods` | out | number of periods n (may be fractional) | , |

**Applies when:** present_value > 0; payment > 0; 0 < rate < 1; payment larger than the interest of one period.

```
$ economics-finance solve number_of_periods present_value=10000 payment=500 rate=0.01
periods = 22.42574188
  by: Number of payments to repay a loan  (n = -ln(1 - PV r / PMT) / ln(1 + r) (PMT (1 + r) for
an annuity due))
  present_value = 10000
  payment = 500
  rate = 0.01
  assumes: present_value > 0; payment > 0; 0 < rate < 1; payment larger than the interest of one
period
```

**What the worked example means:** This calculation determines the number of periodic payments required to fully repay a loan. Given a principal amount of 10000, a fixed payment of 500, and an interest rate of 0.01 per period, the model computes that it will take 22.4257 periods to clear the debt. If the timing is set to 'due', payments occur at the beginning of each period, which typically reduces the total time compared to ordinary annuities where payments are made at the end.

#### Remaining balance of a loan, `loan_balance`

The balance still owed on an amortizing loan after k level payments.

$$B_k = P(1+i)^k - PMT\,\frac{(1+i)^k - 1}{i}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount borrowed | , |
| `annual_rate` | in | nominal annual interest rate (decimal), compounded at the payment frequency | , |
| `years` | in | term of the loan in years | , |
| `payments_made` | in | number of payments already made | , |
| `payments_per_year` | in | payments per year (12 = monthly, default) (optional) | , |
| `balance` | out | balance owed after k payments | , |
| `payment` | out | level payment per period | , |

**Applies when:** principal > 0; 0 <= annual_rate < 1; 0 <= payments_made <= number of payments.

```
$ economics-finance solve loan_balance principal=200000 annual_rate=0.06 years=30
payments_made=60
balance = 186108.7136
payment = 1199.10105
  by: Remaining balance of a loan  (B_k = P (1 + i)^k - PMT ((1 + i)^k - 1)/i)
  principal = 200000
  annual_rate = 0.06
  years = 30
  assumes: principal > 0; 0 <= annual_rate < 1; 0 <= payments_made <= number of payments
```

**What the worked example means:** After 60 payments, the remaining balance is 186109, with a periodic payment of 1199.1.

### Macroeconomics (8)

#### Inflation from a price index series, `inflation_adjustment`

Total and average annual (geometric) inflation from consecutive price levels (CPI), and the value of an amount in the last period's money expressed in first-period money.

$$\bar{\pi} = \left(\frac{P_n}{P_0}\right)^{1/n} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_levels` | in | price index levels in consecutive periods (e.g. CPI each year) | , |
| `amount` | in | an amount in last-period money to deflate (optional) (optional) | , |
| `total_inflation` | out | P_last / P_first - 1 | , |
| `average_inflation` | out | (P_last/P_first)^(1/n) - 1 | , |
| `real_value` | out | amount x P_first / P_last | , |

**Applies when:** price levels > 0.

```
$ economics-finance solve inflation_adjustment "price_levels=[100, 103, 107.12]" amount=1000
total_inflation = 0.0712
average_inflation = 0.03498792263
real_value = 933.5324869
  by: Inflation from a price index series  (geometric mean of the period-to-period price
relatives)
  amount = 1000
  assumes: price levels > 0
```

**What the worked example means:** This calculation determines the total change in price levels from the first to the last period, resulting in 0.0712. It also computes the geometric mean annual inflation rate, which is 0.0349879, representing the constant yearly growth that would yield the same final price level. Additionally, it converts the specified nominal amount into purchasing power equivalent to the base period, yielding a real value of 933.532.

**Keynesian spending multiplier, `spending_multiplier`**

Government-spending multiplier 1/(1 - mpc(1 - t) + m), the lump-sum tax multiplier and the change in output from a change in spending.

$$k = \frac{1}{1 - c(1 - t) + m}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mpc` | in | marginal propensity to consume | , |
| `tax_rate` | in | proportional income tax rate (default 0) (optional) | , |
| `import_propensity` | in | marginal propensity to import (default 0) (optional) | , |
| `spending_change` | in | change in government spending (optional) (optional) | , |
| `spending_multiplier` | out | 1/(1 - mpc(1 - t) + m) | , |
| `tax_multiplier` | out | -mpc x multiplier | , |
| `output_change` | out | multiplier x spending change | , |

**Applies when:** 0 < mpc < 1; 0 <= mpc(1 - t) - m < 1.

```
$ economics-finance solve spending_multiplier mpc=0.8 tax_rate=0.25 import_propensity=0.1
spending_change=100
spending_multiplier = 2
tax_multiplier = -1.6
output_change = 200
  by: Keynesian spending multiplier  (k = 1/(1 - mpc(1 - t) + m))
  mpc = 0.8
  tax_rate = 0.25
  import_propensity = 0.1
  spending_change = 100
  assumes: 0 < mpc < 1; 0 <= mpc(1 - t) - m < 1
```

**What the worked example means:** This calculation determines the Keynesian spending multiplier using the marginal propensity to consume, tax rate, and import propensity. The resulting multiplier of 2 indicates how much total income rises for each unit of government spending. It also derives the tax multiplier of -1.6 and the predicted change in output of 200 based on the specified spending shift.

**Laspeyres, Paasche and Fisher price indices, `price_indices`**

Price indices (base = 100) of a basket: Laspeyres (base-year quantities, the CPI method), Paasche (current quantities) and Fisher (their geometric mean).

$$L = 100\frac{\sum p_1 q_0}{\sum p_0 q_0}, \quad P = 100\frac{\sum p_1 q_1}{\sum p_0 q_1}, \quad F = \sqrt{LP}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `base_prices` | in | prices in the base year | , |
| `base_quantities` | in | quantities in the base year | , |
| `current_prices` | in | prices in the current year | , |
| `current_quantities` | in | quantities in the current year | , |
| `laspeyres` | out | 100 sum p1 q0 / sum p0 q0 | , |
| `paasche` | out | 100 sum p1 q1 / sum p0 q1 | , |
| `fisher` | out | sqrt(L P) | , |

**Applies when:** one price and quantity per good; prices > 0.

```
$ economics-finance solve price_indices "base_prices=[2, 5]" "base_quantities=[100, 50]"
"current_prices=[3, 6]" "current_quantities=[150, 40]"
laspeyres = 133.3333333
paasche = 138
fisher = 135.6465997
  by: Laspeyres, Paasche and Fisher price indices  (ratios of basket costs)
  assumes: one price and quantity per good; prices > 0
```

**What the worked example means:** This calculation determines three price indices based on a basket of goods. The Laspeyres index, valued at 133.333, uses base year quantities to measure cost changes. The Paasche index, valued at 138, uses current year quantities. The Fisher index, valued at 135.647, is the geometric mean of these two, offering a compromise between the upward bias of Laspeyres and the downward bias of Paasche.

**Money multiplier, `money_multiplier`**

Simple deposit multiplier 1/rr and the money multiplier (1 + c)/(rr + c) with a currency-deposit ratio c, and the money supply from the monetary base.

$$m = \frac{1 + c}{rr + c}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `reserve_ratio` | in | reserve ratio (decimal) | , |
| `currency_ratio` | in | currency-to-deposit ratio (default 0) (optional) | , |
| `monetary_base` | in | monetary base (optional) (optional) | , |
| `simple_multiplier` | out | 1/rr | , |
| `money_multiplier` | out | (1 + c)/(rr + c) | , |
| `money_supply` | out | multiplier x base | , |

**Applies when:** 0 < reserve_ratio <= 1; currency_ratio >= 0.

```
$ economics-finance solve money_multiplier reserve_ratio=0.1 currency_ratio=0.2
monetary_base=1000
simple_multiplier = 10
money_multiplier = 4
money_supply = 4000
  by: Money multiplier  (m = (1 + c)/(rr + c))
  reserve_ratio = 0.1
  currency_ratio = 0.2
  monetary_base = 1000
  assumes: 0 < reserve_ratio <= 1; currency_ratio >= 0
```

**What the worked example means:** This calculation determines the money multiplier using the formula involving the reserve ratio and the currency-to-deposit ratio. With a reserve ratio of 0.1 and a currency ratio of 0.2, the resulting multiplier is 4. If a monetary base of 1000 is provided, the estimated money supply becomes 4000. This figure indicates how much broad money can be generated for every unit of high-powered money in the system.

### Nominal GDP, real GDP and the GDP deflator, `gdp_deflator`

Nominal GDP (current prices), real GDP (base-year prices), the GDP deflator, inflation since the base year and real growth, from the prices and quantities of the goods produced.

$$\text{Deflator} = 100 \, \frac{\sum p_1 q_1}{\sum p_0 q_1}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `base_prices` | in | prices in the base year | , |
| `base_quantities` | in | quantities in the base year | , |
| `current_prices` | in | prices in the current year | , |
| `current_quantities` | in | quantities in the current year | , |
| `nominal_gdp` | out | sum p1 q1 | , |
| `real_gdp` | out | sum p0 q1 | , |
| `deflator` | out | 100 x nominal / real | , |
| `inflation` | out | deflator/100 - 1 (since the base year) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `real_growth` | out | real GDP / base-year GDP - 1 | , |

**Applies when:** one price and quantity per good; prices > 0.

```
$ economics-finance solve gdp_deflator "base_prices=[2, 5]" "base_quantities=[100, 50]"
"current_prices=[3, 6]" "current_quantities=[120, 60]"
nominal_gdp = 720
real_gdp = 540
deflator = 133.3333333
inflation = 0.3333333333
real_growth = 0.2
  by: Nominal GDP, real GDP and the GDP deflator  (deflator = 100 sum p1 q1 / sum p0 q1)
  assumes: one price and quantity per good; prices > 0
```

**What the worked example means:** This calculation determines the GDP deflator by comparing nominal GDP, which uses current prices and quantities, to real GDP, which uses base-year prices and current quantities. The resulting deflator value of 133.333 reflects the overall price level change since the base year. This metric makes it possible to isolate inflation effects from actual production growth, showing that output measured in constant dollars grew by 0.2.

**Real interest rate (Fisher equation), `fisher_real_rate`**

The exact real rate (1 + i)/(1 + pi) - 1 and the approximation i - pi.

$$1 + i = (1 + r)(1 + \pi)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `nominal_rate` | in | nominal interest rate (decimal) | , |
| `inflation_rate` | in | inflation rate (decimal) | , |
| `real_rate` | out | (1 + i)/(1 + pi) - 1 | , |
| `approximate_real_rate` | out | i - pi | , |

**Applies when:** -1 < rates < 1.

```
$ economics-finance solve fisher_real_rate nominal_rate=0.08 inflation_rate=0.03
real_rate = 0.04854368932
approximate_real_rate = 0.05
  by: Real interest rate (Fisher equation)  ((1 + i) = (1 + r)(1 + pi))
  nominal_rate = 0.08
  inflation_rate = 0.03
  assumes: -1 < rates < 1
```

**What the worked example means:** This calculation determines the true return on an investment by adjusting the nominal interest rate for inflation. Using a nominal rate of 0.08 and an inflation rate of 0.03, the exact real rate is computed as 0.0485437, while the simpler approximation yields 0.05. The exact method preserves purchasing power more accurately than the linear subtraction, which is useful for quick estimates but less precise at higher rates.

**Solow model steady state, `solow_steady_state`**

Steady-state capital, output and consumption per effective worker in the Solow model with y = A k^alpha, and the golden-rule saving rate and capital.

$$k^* = \left( \frac{sA}{n + g + \delta} \right)^{\frac{1}{1-\alpha}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `saving_rate` | in | saving rate s (decimal) | , |
| `population_growth` | in | population growth n (decimal) | , |
| `depreciation` | in | depreciation rate (decimal) | , |
| `alpha` | in | capital share alpha in y = A k^alpha | , |
| `technology_growth` | in | technology growth g (default 0) (optional) | , |
| `productivity` | in | total factor productivity A (default 1) (optional) | , |
| `capital` | out | k* = (sA/(n + g + delta))^(1/(1 - alpha)) | , |
| `output` | out | y* = A k*^alpha | , |
| `consumption` | out | (1 - s) y* | , |
| `golden_rule_saving` | out | alpha | , |
| `golden_rule_capital` | out | (alpha A/(n + g + delta))^(1/(1 - alpha)) | , |

**Applies when:** 0 < s, alpha < 1; n + g + delta > 0.

```
$ economics-finance solve solow_steady_state saving_rate=0.2 population_growth=0.01
depreciation=0.05 alpha=0.3
capital = 5.584311504
output = 1.675293451
consumption = 1.340234761
golden_rule_saving = 0.3
golden_rule_capital = 9.966176578
  by: Solow model steady state  (s A k^alpha = (n + g + delta) k)
  saving_rate = 0.2
  population_growth = 0.01
  depreciation = 0.05
  alpha = 0.3
  assumes: 0 < s, alpha < 1; n + g + delta > 0
```

**What the worked example means:** Steady state capital per effective worker is 5.58431, yielding output of 1.67529 and consumption of 1.34023.

### Unemployment rate and labour-force participation, `unemployment_rate`

Labour force, unemployment rate, and (with the working-age population) the labour-force participation rate, the employment-population ratio and the number not in the labour force.

$$u = \frac{U}{E + U}, \qquad \text{participation} = \frac{E + U}{N}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `employed` | in | number of employed people | , |
| `unemployed` | in | number of unemployed people (without work and looking for it) | , |
| `working_age_population` | in | working-age (adult) population (optional) (optional) | , |
| `labour_force` | out | employed + unemployed | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `unemployment_rate` | out | unemployed / labour force | , |
| `participation_rate` | out | labour force / working-age population | , |
| `employment_population_ratio` | out | employed / working-age population | , |
| `not_in_labour_force` | out | working-age population - labour force | , |

**Applies when:** employed, unemployed >= 0, labour force > 0; labour force <= working-age population.

```
$ economics-finance solve unemployment_rate employed=150 unemployed=10 working_age_population=250
labour_force = 160
unemployment_rate = 0.0625
participation_rate = 0.64
employment_population_ratio = 0.6
not_in_labour_force = 90
  by: Unemployment rate and labour-force participation  (LF = E + U; u = U / LF; participation =
LF / N; employment ratio = E / N)
  employed = 150
  unemployed = 10
  working_age_population = 250
  assumes: employed, unemployed >= 0, labour force > 0; labour force <= working-age population
```

In the worked example, employed = 150, unemployed = 10, working_age_population = 250 go through the equation above and give labour_force = 160, unemployment_rate = 0.0625, participation_rate = 0.64, employment_population_ratio = 0.6. It applies only when employed, unemployed >= 0, labour force > 0; labour force <= working-age population: outside that it says what is wrong instead of returning a number.

## Microeconomics (14)

### Break-even and target-profit volume, `break_even`

Units to sell to cover fixed costs (and reach a target profit): Q = (F + target) / (p - v).

$$Q_{BE} = \frac{F + \pi_{target}}{p - v}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `fixed_costs` | in | total fixed costs | , |
| `price` | in | selling price per unit | , |
| `variable_cost` | in | variable cost per unit | , |
| `target_profit` | in | target profit (default 0: break-even) (optional) | , |
| `quantity` | out | units (F + target)/(p - v) | , |
| `revenue` | out | p Q | , |
| `contribution_margin` | out | p - v per unit | , |
| `contribution_margin_ratio` | out | (p - v)/p | , |

**Applies when:** price > variable cost; fixed costs >= 0.

```
$ economics-finance solve break_even fixed_costs=10000 price=50 variable_cost=30
quantity = 500
revenue = 25000
contribution_margin = 20
contribution_margin_ratio = 0.4
  by: Break-even and target-profit volume  (Q = (F + target) / (p - v))
  fixed_costs = 10000
  price = 50
  variable_cost = 30
  assumes: price > variable cost; fixed costs >= 0
```

**What the worked example means:** This calculation determines the break-even or target-profit volume by dividing total fixed costs plus the desired profit by the contribution margin per unit. With fixed costs of 10000, a selling price of 50, and variable costs of 30, each unit contributes 20 toward covering expenses. Consequently, the required sales volume is 500 units, generating total revenue of 25000. The contribution margin ratio stands at 0.4, indicating how much of each dollar of sales remains after covering variable costs.

**Cost curves, best output and shutdown of a competitive firm, `competitive_firm_cost_curves`**

Fixed, average fixed, average variable, average total and marginal cost from a total cost table, and the price-taking firm's profit-maximising output, its profit, the shutdown price (minimum AVC) and the break-even price (minimum ATC).

$$\mathrm{AVC} = \frac{TC - FC}{Q}, \quad \mathrm{MC} = \frac{\Delta TC}{\Delta Q}, \quad Q^{*} = \arg\max_{Q} \left(PQ - TC(Q)\right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `quantities` | in | output levels of the table, starting at 0 and increasing | , |
| `total_costs` | in | total cost at each output level (the first is the fixed cost) | , |
| `price` | in | market price the firm takes as given | , |
| `fixed_cost` | out | total cost at output 0 | , |
| `average_fixed_cost` | out | AFC = FC / Q for each output > 0 | , |
| `average_variable_cost` | out | AVC = (TC - FC) / Q for each output > 0 | , |
| `average_total_cost` | out | ATC = TC / Q for each output > 0 | , |
| `marginal_cost` | out | MC = change in TC / change in Q for each step of the table | , |
| `best_output` | out | profit-maximising output in the table (0 = shut down) | , |
| `profit` | out | profit P Q - TC at the best output (negative = loss) | , |
| `shutdown_price` | out | minimum AVC: below it the firm produces nothing | , |
| `break_even_price` | out | minimum ATC: the price at which the best profit is zero | , |
| `decision` | out | produce or shut down | , |

**Applies when:** at least 3 rows, one total cost per output; first output 0, outputs increasing; total costs non-negative and non-decreasing; price > 0.

```
$ economics-finance solve competitive_firm_cost_curves "quantities=[0, 1, 2, 3, 4, 5, 6]"
"total_costs=[10, 18, 24, 32, 44, 60, 80]" price=16
fixed_cost = 10
average_fixed_cost = [10.0,5.0,3.3333333333333335,2.5,2.0,1.6666666666666667]
average_variable_cost = [8.0,7.0,7.333333333333333,8.5,10.0,11.666666666666666]
average_total_cost = [18.0,12.0,10.666666666666666,11.0,12.0,13.333333333333334]
marginal_cost = [8.0,6.0,8.0,12.0,16.0,20.0]
best_output = 5
profit = 20
shutdown_price = 7
break_even_price = 10.66666667
decision = produce
  by: Cost curves, best output and shutdown of a competitive firm  (AFC = FC/Q, AVC = (TC -
FC)/Q, ATC = TC/Q, MC = dTC/dQ; best output = the row maximising P Q - TC (ties: larger output);
shutdown price = min AVC, break-even price = min ATC)
  price = 16
  assumes: at least 3 rows, one total cost per output; first output 0, outputs increasing; total
costs non-negative and non-decreasing; price > 0
```

In the worked example, quantities = [0, 1, 2, 3, 4, 5, 6], total_costs = [10, 18, 24, 32, 44, 60, 80], price = 16 go through the equation above and give fixed_cost = 10, average_fixed_cost = [10, 5, 3.333333333, 2.5, 2, 1.666666667], average_variable_cost = [8, 7, 7.333333333, 8.5, 10, 11.66666667], average_total_cost = [18, 12, 10.66666667, 11, 12, 13.33333333]. It applies only when at least 3 rows, one total cost per output; first output 0, outputs increasing; total costs non-negative and non-decreasing; price > 0: outside that it says what is wrong instead of returning a number.

**Cournot duopoly equilibrium, `cournot`**

Quantities, price and profits of two firms choosing quantities simultaneously against P = A - B(q1 + q2) with constant marginal costs.

$$q_i^* = \frac{A - 2c_i + c_j}{3B}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_intercept` | in | A in the inverse demand P = A - B Q | , |
| `price_slope` | in | B in P = A - B Q (B > 0) | , |
| `cost_1` | in | marginal cost of firm 1 | , |
| `cost_2` | in | marginal cost of firm 2 | , |
| `q1` | out | (A - 2c1 + c2)/(3B) | , |
| `q2` | out | (A - 2c2 + c1)/(3B) | , |
| `price` | out | A - B(q1 + q2) | , |
| `profit_1` | out | (P - c1) q1 | , |
| `profit_2` | out | (P - c2) q2 | , |

**Applies when:** B > 0; both equilibrium quantities positive.

```
$ economics-finance solve cournot price_intercept=100 price_slope=1 cost_1=10 cost_2=10
q1 = 30
q2 = 30
price = 40
profit_1 = 900
profit_2 = 900
  by: Cournot duopoly equilibrium  (intersection of the two best-response functions)
  price_intercept = 100
  price_slope = 1
  cost_1 = 10
  cost_2 = 10
  assumes: B > 0; both equilibrium quantities positive
```

**What the worked example means:** This calculation determines the stable production levels for two firms competing simultaneously under linear demand. Solving the intersection of their best-response functions shows that firm one produces 30 and firm two produces 30. These quantities lead to a market price of 40, yielding profits of 900 for firm one and 900 for firm two.

**Cross-price elasticity of demand (midpoint), `cross_price_elasticity`**

Midpoint cross-price elasticity: response of the quantity of good A to the price of good B; positive = substitutes, negative = complements.

$$E_{AB} = \frac{\Delta Q_A / \bar{Q}_A}{\Delta P_B / \bar{P}_B}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_other_1` | in | first price of the other good | , |
| `quantity_1` | in | quantity of this good at that price | , |
| `price_other_2` | in | second price of the other good | , |
| `quantity_2` | in | quantity of this good at the second price | , |
| `elasticity` | out | (dQa / mean Qa) / (dPb / mean Pb) | , |
| `relationship` | out | substitutes, complements or independent | , |

**Applies when:** prices and quantities > 0; the prices differ.

```
$ economics-finance solve cross_price_elasticity price_other_1=2 quantity_1=100 price_other_2=2.5
quantity_2=120
elasticity = 0.8181818182
relationship = substitutes
  by: Cross-price elasticity of demand (midpoint)  (midpoint formula)
  price_other_1 = 2
  quantity_1 = 100
  price_other_2 = 2.5
  quantity_2 = 120
  assumes: prices and quantities > 0; the prices differ
```

**What the worked example means:** This calculation determines the midpoint cross-price elasticity, which measures how the quantity demanded of one good responds to a change in the price of another. Using the initial price of the other good at 2 and its corresponding quantity of this good at 100, alongside the second price of the other good at 2.5 and the new quantity at 120, gives the percentage changes relative to their averages. The resulting elasticity

value is 0.818182. A positive value suggests the goods are substitutes, meaning an increase in the price of one leads to higher demand for the other, while a negative value indicates they are complements.

**Gini coefficient, `gini_coefficient`**

Gini coefficient of a list of incomes (population formula: mean absolute difference / twice the mean; 0 = equality, (n-1)/n = one person has everything).

$$G = \frac{\sum_i \sum_j |x_i - x_j|}{2n^2 \bar{x}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `incomes` | in | incomes (or wealth) of the people or households | , |
| `gini` | out | sum (2i - n - 1) x_(i) / (n sum x) | , |

**Applies when:** incomes >= 0 with a positive total.

```
$ economics-finance solve gini_coefficient "incomes=[20000, 35000, 50000, 120000]"
gini = 0.35
  by: Gini coefficient  (sorted-rank formula)
  assumes: incomes >= 0 with a positive total
```

**What the worked example means:** This calculation determines the Gini coefficient, a measure of statistical dispersion intended to represent income or wealth inequality within a nation or group. Using the sorted-rank formula on the input incomes, which are assumed to be non-negative with a positive total, the resulting value is 0.35. A score near zero suggests perfect equality where everyone has the same income, while a higher value indicates greater disparity among the individuals in the dataset.

**Herfindahl-Hirschman index and concentration ratio, `herfindahl_index`**

Market concentration from the firms' shares or sales (normalized to shares): HHI on the 0-10,000 scale, the four-firm concentration ratio and the equivalent number of equal-sized firms.

$$HHI = 10\,000 \sum_i s_i^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `market_shares` | in | market shares (in %) or sales of each firm | , |
| `hhi` | out | 10,000 x sum of squared shares | , |
| `hhi_fraction` | out | sum of squared shares (0-1) | , |
| `cr4` | out | combined share of the four largest firms, % | , |
| `equivalent_firms` | out | 1 / HHI (fraction) | , |

**Applies when:** shares or sales >= 0 with a positive total.

```
$ economics-finance solve herfindahl_index "market_shares=[40, 30, 20, 10]"
hhi = 3000
hhi_fraction = 0.3
cr4 = 100
equivalent_firms = 3.333333333
  by: Herfindahl-Hirschman index and concentration ratio  (normalize to shares s_i; HHI = sum
s_i^2)
  assumes: shares or sales >= 0 with a positive total
```

**What the worked example means:** This calculation determines market concentration using the Herfindahl-Hirschman Index, which sums the squares of individual firm shares scaled to ten thousand. The resulting index value of 3000 corresponds to a fraction of 0.3, while the top four firms hold a combined share of 100. An equivalent number of equal-sized firms would be 3.33333, providing an intuitive sense of how many competitors would produce the same level of concentration if they were all identical in size.

**Income elasticity of demand (midpoint),** `income_elasticity`

Midpoint income elasticity of demand and the good's type (inferior, necessity, luxury).

$$E_I = \frac{(Q_2 - Q_1)/\bar{Q}}{(I_2 - I_1)/\bar{I}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `income_1` | in | first income | , |
| `quantity_1` | in | quantity at income 1 | , |
| `income_2` | in | second income | , |
| `quantity_2` | in | quantity at income 2 | , |
| `elasticity` | out | (dQ / mean Q) / (dI / mean I) | , |
| `classification` | out | inferior, necessity or luxury | , |

**Applies when:** incomes and quantities > 0; the incomes differ.

```
$ economics-finance solve income_elasticity income_1=40000 quantity_1=10 income_2=50000
quantity_2=13
elasticity = 1.173913043
classification = normal good (luxury)
  by: Income elasticity of demand (midpoint)  (midpoint formula)
  income_1 = 40000
  quantity_1 = 10
  income_2 = 50000
  quantity_2 = 13
  assumes: incomes and quantities > 0; the incomes differ
```

**What the worked example means:** This calculation determines how responsive the quantity demanded of a good is to changes in consumer income using the midpoint method. Comparing the percentage change in quantity to the percentage change in income, relative to their average values, gives an elasticity of 1.17391. This value indicates that the good is classified as a normal good (luxury), meaning consumers adjust their purchasing habits in this specific way when their financial situation changes.

**Market equilibrium and surplus (linear curves),** `market_equilibrium`

Equilibrium price and quantity of Qd = a - bP and Qs = c + dP, with consumer, producer and total surplus (producer surplus counted at prices >= 0).

$$P^* = \frac{a - c}{b + d}, \quad CS = \frac{Q^{*2}}{2b}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `demand_intercept` | in | a in Qd = a - bP | , |
| `demand_slope` | in | b in Qd = a - bP (b > 0) | , |
| `supply_intercept` | in | c in Qs = c + dP | , |
| `supply_slope` | in | d in Qs = c + dP (d > 0) | , |
| `price` | out | equilibrium price (a - c)/(b + d) | , |
| `quantity` | out | equilibrium quantity | , |
| `consumer_surplus` | out | Q^2 / (2b) | , |
| `producer_surplus` | out | area above supply, below P | , |
| `total_surplus` | out | CS + PS | , |

**Applies when:** b, d > 0; positive equilibrium price and quantity.

```
$ economics-finance solve market_equilibrium demand_intercept=100 demand_slope=2
supply_intercept=-20 supply_slope=4
price = 20
quantity = 60
consumer_surplus = 900
producer_surplus = 450
total_surplus = 1350
  by: Market equilibrium and surplus (linear curves)  (a - bP = c + dP; surpluses are
triangle/trapezoid areas)
  demand_intercept = 100
  demand_slope = 2
  supply_intercept = -20
  supply_slope = 4
  assumes: b, d > 0; positive equilibrium price and quantity
```

**What the worked example means:** This calculation determines the market clearing point where linear demand and supply intersect. Using the input parameters, the model computes an equilibrium price of 20 and a quantity of 60. Consumer surplus is found to be 900 and producer surplus is 450, resulting in a combined welfare measure of 1350. These values represent the efficiency gains from trade at this specific price and quantity level.

### Monopoly profit maximization, `monopoly`

Profit-maximising quantity and price of a monopoly facing P = A - BQ with cost F + cQ + eQ^2, its profit, Lerner index and the deadweight loss against marginal-cost pricing.

$$Q^* = \frac{A - c}{2(B + e)}, \quad P^* = A - BQ^*$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_intercept` | in | A in the inverse demand P = A - B Q | , |
| `price_slope` | in | B in P = A - B Q (B > 0) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `marginal_cost` | in | c, the constant part of marginal cost | , |
| `fixed_cost` | in | fixed cost F (default 0) (optional) | , |
| `cost_quadratic` | in | e in the cost F + cQ + eQ^2 (default 0) (optional) | , |
| `quantity` | out | Q* where MR = MC | , |
| `price` | out | A - B Q* | , |
| `profit` | out | P Q - cost | , |
| `lerner_index` | out | (P - MC)/P | , |
| `deadweight_loss` | out | loss against P = MC | , |

**Applies when:** A > c >= 0; B > 0; F, e >= 0.

```
$ economics-finance solve monopoly price_intercept=100 price_slope=2 marginal_cost=20
quantity = 20
price = 60
profit = 800
lerner_index = 0.6666666667
deadweight_loss = 400
  by: Monopoly profit maximization  (MR = A - 2BQ = MC = c + 2eQ)
  price_intercept = 100
  price_slope = 2
  marginal_cost = 20
  assumes: A > c >= 0; B > 0; F, e >= 0
```

**What the worked example means:** The profit-maximizing quantity is 20 and the price is 60, yielding a profit of 800.

**Per-unit tax incidence (linear curves),** `per_unit_tax`

Effect of a per-unit tax on Qd = a - bP, Qs = c + dP: quantity, price paid by buyers and kept by sellers, tax revenue, deadweight loss and the buyers' share of the burden.

$$P_b = \frac{a - c + d\,t}{b + d}, \quad DWL = \tfrac{1}{2}\,t\,(Q_0 - Q_t)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `demand_intercept` | in | a in Qd = a - bP | , |
| `demand_slope` | in | b in Qd = a - bP (b > 0) | , |
| `supply_intercept` | in | c in Qs = c + dP | , |
| `supply_slope` | in | d in Qs = c + dP (d > 0) | , |
| `tax` | in | tax per unit | , |
| `quantity` | out | quantity with the tax | , |
| `buyer_price` | out | price paid by buyers | , |
| `seller_price` | out | buyer price - tax | , |
| `tax_revenue` | out | t Q | , |
| `deadweight_loss` | out | t (Q0 - Qt) / 2 | , |
| `buyer_share` | out | share of the tax borne by buyers d/(b+d) | , |

**Applies when:** b, d > 0; tax > 0; positive quantity and seller price with the tax.

```
$ economics-finance solve per_unit_tax demand_intercept=100 demand_slope=2 supply_intercept=-20
supply_slope=4 tax=3
quantity = 56
buyer_price = 22
seller_price = 19
tax_revenue = 168
deadweight_loss = 6
buyer_share = 0.6666666667
  by: Per-unit tax incidence (linear curves)  (a - b Pb = c + d (Pb - t))
  demand_intercept = 100
  demand_slope = 2
  supply_intercept = -20
  supply_slope = 4
  tax = 3
  assumes: b, d > 0; tax > 0; positive quantity and seller price with the tax
```

In the worked example, demand_intercept = 100, demand_slope = 2, supply_intercept = -20, supply_slope = 4, tax = 3 go through the equation above and give quantity = 56, buyer_price = 22, seller_price = 19, tax_revenue = 168. It applies only when b, d > 0; tax > 0; positive quantity and seller price with the tax: outside that it says what is wrong instead of returning a number.

**Point elasticity of a linear demand curve, `point_elasticity`**

Quantity, point price elasticity and revenue on the linear demand curve Q = a - b P at a price, and the revenue-maximising price.

$$E = \frac{dQ}{dP}\frac{P}{Q} = \frac{-bP}{a - bP}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `demand_intercept` | in | a in Q = a - bP | , |
| `demand_slope` | in | b in Q = a - bP (b > 0) | , |
| `price` | in | price at which to evaluate | , |
| `quantity` | out | Q = a - bP | , |
| `elasticity` | out | -b P / Q | , |
| `revenue` | out | P Q | , |
| `revenue_maximizing_price` | out | a / (2b) | , |

**Applies when:** a, b, P > 0; Q = a - bP > 0.

```
$ economics-finance solve point_elasticity demand_intercept=100 demand_slope=2 price=20
quantity = 60
elasticity = -0.6666666667
revenue = 1200
revenue_maximizing_price = 25
  by: Point elasticity of a linear demand curve  (E = (dQ/dP)(P/Q) = -b P / (a - bP))
  demand_intercept = 100
  demand_slope = 2
  price = 20
  assumes: a, b, P > 0; Q = a - bP > 0
```

**What the worked example means:** This calculation determines the economic metrics for a linear demand curve defined by an intercept and slope. Given the specified price, the resulting quantity sold is 60. The point price

elasticity at this specific location on the curve is calculated as -0.666667, indicating how responsive consumers are to price changes here. Additionally, the total revenue generated at this price is 1200, while the price that would theoretically maximize this revenue is identified as 25.

**Price elasticity of demand (midpoint),** `arc_price_elasticity`

Arc (midpoint) price elasticity of demand between two price-quantity points, and whether demand is elastic, inelastic or unit elastic there.

$$E = \frac{(Q_2 - Q_1)/[(Q_1 + Q_2)/2]}{(P_2 - P_1)/[(P_1 + P_2)/2]}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_1` | in | first price | , |
| `quantity_1` | in | quantity demanded at price 1 | , |
| `price_2` | in | second price | , |
| `quantity_2` | in | quantity demanded at price 2 | , |
| `elasticity` | out | (dQ / mean Q) / (dP / mean P) | , |
| `abs_elasticity` | out | \|elasticity\| | , |
| `classification` | out | elastic, inelastic, unit elastic or perfectly inelastic | , |

**Applies when:** prices and quantities > 0; the two prices differ.

```
$ economics-finance solve arc_price_elasticity price_1=10 quantity_1=100 price_2=12 quantity_2=80
elasticity = -1.222222222
abs_elasticity = 1.222222222
classification = elastic
  by: Price elasticity of demand (midpoint)  (midpoint formula: percentage changes relative to
the averages)
  price_1 = 10
  quantity_1 = 100
  price_2 = 12
  quantity_2 = 80
  assumes: prices and quantities > 0; the two prices differ
```

**What the worked example means:** This calculation determines the arc price elasticity of demand between two points using the midpoint method. It compares the percentage change in quantity demanded from 100 to 80 against the percentage change in price from 10 to 12, both relative to their averages. The resulting elasticity of -1.22222 indicates that demand is elastic, meaning consumers are relatively responsive or unresponsive to price changes over this interval.

**Stackelberg duopoly equilibrium,** `stackelberg`

Quantities, price and profits when a leader chooses its quantity first and the follower best-responds, against P = A - B(qL + qF) with constant marginal costs.

$$q_L^* = \frac{A - 2c_L + c_F}{2B}, \quad q_F^* = \frac{A - c_F - Bq_L^*}{2B}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_intercept` | in | A in the inverse demand P = A - B Q | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price_slope` | in | B in P = A - B Q (B > 0) | , |
| `cost_leader` | in | marginal cost of the leader | , |
| `cost_follower` | in | marginal cost of the follower | , |
| `q_leader` | out | (A - 2cL + cF)/(2B) | , |
| `q_follower` | out | (A - cF - B qL)/(2B) | , |
| `price` | out | market price | , |
| `profit_leader` | out | leader's profit | , |
| `profit_follower` | out | follower's profit | , |

**Applies when:** B > 0; both quantities positive.

```
$ economics-finance solve stackelberg price_intercept=100 price_slope=1 cost_leader=10
cost_follower=10
q_leader = 45
q_follower = 22.5
price = 32.5
profit_leader = 1012.5
profit_follower = 506.25
  by: Stackelberg duopoly equilibrium  (backward induction: follower's best response, then the
leader's optimum)
  price_intercept = 100
  price_slope = 1
  cost_leader = 10
  cost_follower = 10
  assumes: B > 0; both quantities positive
```

**What the worked example means:** This calculation determines the equilibrium quantities for a Stackelberg duopoly using backward induction. The leader sets its quantity first, anticipating the follower's best response. Given the inverse demand curve and constant marginal costs, the leader optimizes its profit by choosing a quantity of 45. The follower then observes this move and selects a quantity of 22.5 to maximize its own profit. These decisions result in a total market output that clears at a price of 32.5. The leader typically captures a larger share of the market and higher profits than the follower due to the first-mover advantage in quantity competition.

### Utility-maximising demand (Cobb-Douglas), `cobb_douglas_demand`

The bundle that maximises U = x^alpha y^beta subject to px x + py y = M.

$$x^* = \frac{\alpha}{\alpha+\beta}\frac{M}{p_x}, \quad y^* = \frac{\beta}{\alpha+\beta}\frac{M}{p_y}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `alpha` | in | exponent of good x | , |
| `beta` | in | exponent of good y | , |
| `income` | in | income (budget) | , |
| `price_x` | in | price of good x | , |
| `price_y` | in | price of good y | , |
| `x` | out | alpha/(alpha+beta) M/px | , |
| `y` | out | beta/(alpha+beta) M/py | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `utility` | out | x^alpha y^beta | , |
| `share_x` | out | budget share of x | , |

**Applies when:** alpha, beta, income, prices > 0.

```
$ economics-finance solve cobb_douglas_demand alpha=0.3 beta=0.7 income=1000 price_x=10
price_y=20
x = 30
y = 35
utility = 33.41827338
share_x = 0.3
  by: Utility-maximising demand (Cobb-Douglas)  (tangency MRS = px/py with the budget line)
  alpha = 0.3
  beta = 0.7
  income = 1000
  price_x = 10
  price_y = 20
  assumes: alpha, beta, income, prices > 0
```

**What the worked example means:** This calculation finds the best combination of two goods for a consumer with income 1000 facing prices 10 and 20. Using preferences defined by exponents 0.3 and 0.7, the model determines that the consumer should buy 30 units of good x and 35 units of good y. This choice maximizes satisfaction, resulting in a utility level of 33.4183. The result shows that spending on good x represents 0.3 of the total budget.

## Portfolio theory (12)

### Arithmetic and geometric mean return, `mean_returns`

Arithmetic mean, geometric (compound) mean and cumulative return of a series of periodic returns.

$$\bar{R}_G = \left[\prod_{t=1}^{n}(1+R_t)\right]^{1/n} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `returns` | in | periodic returns (decimals) | , |
| `arithmetic_mean` | out | sum R / n | , |
| `geometric_mean` | out | (prod (1 + R))^(1/n) - 1 | , |
| `cumulative_return` | out | prod (1 + R) - 1 | , |

**Applies when:** each return between -100% and 100%.

```
$ economics-finance solve mean_returns "returns=[0.1, -0.05, 0.2]"
arithmetic_mean = 0.08333333333
geometric_mean = 0.07836515339
cumulative_return = 0.254
  by: Arithmetic and geometric mean return  (geometric mean = exp(mean ln(1 + R)) - 1)
  assumes: each return between -100% and 100%
```

**What the worked example means:** This calculation determines the average performance of an investment over a series of periods using both simple averaging and compounding methods. The arithmetic mean, calculated as 0.0833333, represents the simple average of the periodic returns provided in [0.1, -0.05, 0.2]. The geometric mean,

shown as 0.0783652, reflects the compound annual growth rate that would result in the same final value if applied consistently over the period. Additionally, the total wealth change over all periods is captured by the cumulative return, which equals 0.254. These metrics help distinguish between expected single-period performance and actual multi-period wealth accumulation.

**Beta of an asset from return series, `beta_regression`**

Beta (and alpha, correlation, R^2) of an asset from paired asset and market returns: the slope of the ordinary least-squares regression of asset on market returns.

$$\beta = \frac{\mathrm{Cov}(R_i, R_m)}{\mathrm{Var}(R_m)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `asset_returns` | in | returns of the asset (decimals), one per period | , |
| `market_returns` | in | returns of the market in the same periods | , |
| `beta` | out | cov(asset, market) / var(market) | , |
| `alpha` | out | intercept mean(asset) - beta mean(market) | , |
| `correlation` | out | correlation of asset and market | , |
| `r_squared` | out | share of variance explained | , |

**Applies when:** same number of observations, at least 3; returns between -100% and 100%; neither series constant.

```
$ economics-finance solve beta_regression "asset_returns=[0.02, -0.01, 0.03, 0.015, -0.02]"
"market_returns=[0.015, -0.005, 0.02, 0.01, -0.015]"
beta = 1.441176471
alpha = -0.0002058823529
correlation = 0.9959020462
r_squared = 0.9918208857
  by: Beta of an asset from return series  (beta = S_xy / S_xx (sample covariance over sample
variance))
  assumes: same number of observations, at least 3; returns between -100% and 100%; neither
series constant
```

**What the worked example means:** This calculation determines the beta coefficient by measuring the covariance between asset returns and market returns, divided by the variance of the market returns. The resulting value of 1.44118 represents the slope of the regression line, showing how much the asset's return changes for each unit change in the market return. A beta greater than one suggests higher volatility than the market, while a value less than one indicates lower volatility.

**Expected return and risk from scenarios, `scenario_return`**

Expected return, variance and standard deviation of an asset whose return depends on scenarios with given probabilities.

$$E[R] = \sum_i p_i R_i, \quad \sigma^2 = \sum_i p_i (R_i - E[R])^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `probabilities` | in | probability of each scenario (adding up to 1) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `returns` | in | return in each scenario (decimals) | , |
| `expected_return` | out | sum p R | , |
| `variance` | out | sum p (R - E)^2 | , |
| `std_dev` | out | sqrt(variance) | , |

**Applies when:** one probability per return; probabilities >= 0 adding up to 1.

```
$ economics-finance solve scenario_return "probabilities=[0.3, 0.5, 0.2]" "returns=[0.2, 0.1,
-0.15]"
expected_return = 0.08
variance = 0.0151
std_dev = 0.1228820573
  by: Expected return and risk from scenarios  (E = sum p_i R_i; Var = sum p_i (R_i - E)^2)
  assumes: one probability per return; probabilities >= 0 adding up to 1
```

**What the worked example means:** This calculation determines the expected return, variance, and standard deviation for an asset based on discrete scenarios. Weighting each scenario's return by its probability gives the mean outcome 0.08. The variance 0.0151 measures the dispersion of returns around this mean, while the standard deviation 0.122882 expresses this risk in the same units as the original returns.

### Global minimum-variance portfolio, `global_min_variance`

Weights (adding up to 1, short sales allowed) of the portfolio with the smallest variance, from a covariance matrix: w = Sigma^-1 1 / (1' Sigma^-1 1).

$$w^* = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top\Sigma^{-1}\mathbf{1}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `covariance` | in | covariance matrix of the asset returns | , |
| `weights` | out | minimum-variance weights (list) | , |
| `variance` | out | portfolio variance | , |
| `std_dev` | out | portfolio sd | , |

**Applies when:** covariance symmetric and positive definite.

```
$ economics-finance solve global_min_variance "covariance=[[0.04, 0.006, 0.01], [0.006, 0.09,
0.02], [0.01, 0.02, 0.0625]]"
weights = [0.5475060905244262,0.19040902679830743,0.2620848826772663]
variance = 0.02566354661
std_dev = 0.1601984601
  by: Global minimum-variance portfolio  (Cholesky solve of Sigma x = 1, w = x / sum(x))
  assumes: covariance symmetric and positive definite
```

**What the worked example means:** This calculation determines the asset allocation that minimizes total risk for the given covariance structure. Using the input covariance matrix [0.04 0.006 0.01; 0.006 0.09 0.02; 0.01 0.02 0.0625], the algorithm solves for the optimal weights, resulting in the vector [0.547506, 0.190409, 0.262085]. These weights sum to one and allow for short sales, producing a theoretical portfolio variance of 0.0256635 and a corresponding standard deviation of 0.160198. This represents the lowest possible volatility achievable with these specific assets under the stated assumptions.

**Information ratio and tracking error, `information_ratio`**

Mean active return (portfolio - benchmark) per unit of tracking error (sample sd of active returns).

$$IR = \frac{\overline{R_p - R_b}}{s(R_p - R_b)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `portfolio_returns` | in | portfolio returns (decimals) | , |
| `benchmark_returns` | in | benchmark returns in the same periods | , |
| `information_ratio` | out | mean active / tracking error | , |
| `tracking_error` | out | sample sd of active returns | , |
| `mean_active_return` | out | mean of portfolio - benchmark | , |

**Applies when:** same number of observations, at least 2; active returns not constant.

```
$ economics-finance solve information_ratio "portfolio_returns=[0.03, 0.01, 0.04, 0.02]"
"benchmark_returns=[0.02, 0.015, 0.03, 0.01]"
information_ratio = 0.8333333333
tracking_error = 0.0075
mean_active_return = 0.00625
  by: Information ratio and tracking error  (IR = mean(Rp - Rb) / s(Rp - Rb))
  assumes: same number of observations, at least 2; active returns not constant
```

**What the worked example means:** This calculation determines the Information Ratio by dividing the average excess return over the benchmark by the standard deviation of those excess returns. Using portfolio returns of [0.03, 0.01, 0.04, 0.02] and benchmark returns of [0.02, 0.015, 0.03, 0.01], the resulting mean active return is 0.00625 and the tracking error is 0.0075. The final Information Ratio of 0.833333 indicates how much consistent alpha the manager generates for each unit of relative risk taken, serving as a key metric for evaluating active management skill.

**Minimum-variance portfolio of two assets, `min_variance_two_asset`**

The weights of two assets that minimise the portfolio variance (short sales allowed), and that variance's sd.

$$w_1^* = \frac{\sigma_2^2 - \rho\sigma_1\sigma_2}{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `sd_1` | in | standard deviation of asset 1 (decimal) | , |
| `sd_2` | in | standard deviation of asset 2 (decimal) | , |
| `correlation` | in | correlation between the assets (-1..1) | , |
| `weight_1` | out | w1 = (s2^2 - rho s1 s2) / (s1^2 + s2^2 - 2 rho s1 s2) | , |
| `weight_2` | out | 1 - w1 | , |
| `std_dev` | out | standard deviation of the minimum-variance portfolio | , |

**Applies when:** sd_1, sd_2 > 0; -1 <= correlation <= 1; not (correlation = 1 and sd_1 = sd_2).

```
$ economics-finance solve min_variance_two_asset sd_1=0.2 sd_2=0.3 correlation=0.1
weight_1 = 0.7118644068
weight_2 = 0.2881355932
std_dev = 0.1737912248
  by: Minimum-variance portfolio of two assets  (w1 = (s2^2 - rho s1 s2) / (s1^2 + s2^2 - 2 rho
s1 s2))
  sd_1 = 0.2
  sd_2 = 0.3
  correlation = 0.1
  assumes: sd_1, sd_2 > 0; -1 <= correlation <= 1; not (correlation = 1 and sd_1 = sd_2)
```

**What the worked example means:** This calculation finds the specific weights for two assets that minimize overall risk. By using the standard deviations of asset one (0.2) and asset two (0.3), along with their correlation coefficient (0.1), the model determines the optimal allocation. The result shows that investing 0.711864 in the first asset and 0.288136 in the second yields the lowest possible volatility, measured as 0.173791. This represents the point of maximum diversification benefit where risk is minimized without regard to expected return.

### Portfolio expected return and risk, `portfolio_return_risk`

Expected return w'mu, variance w'Sigma w and standard deviation of a portfolio from its weights, expected returns and covariance matrix.

$$E[R_p] = w^\top \mu, \quad \sigma_p^2 = w^\top \Sigma\, w$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `weights` | in | portfolio weights, adding up to 1 (negative = short) | , |
| `expected_returns` | in | expected return of each asset (decimals) | , |
| `covariance` | in | covariance matrix of the returns [[s11, s12], [s21, s22]] | , |
| `expected_return` | out | w' mu | , |
| `variance` | out | w' Sigma w | , |
| `std_dev` | out | sqrt(variance) | , |

**Applies when:** one weight, return and row per asset; weights add up to 1; covariance symmetric, positive semidefinite.

```
$ economics-finance solve portfolio_return_risk "weights=[0.6, 0.4]" "expected_returns=[0.1,
0.15]" "covariance=[[0.04, 0.006], [0.006, 0.09]]"
expected_return = 0.12
variance = 0.03168
std_dev = 0.1779887637
  by: Portfolio expected return and risk  (E = sum w_i mu_i; Var = sum_i sum_j w_i w_j sigma_ij)
  assumes: one weight, return and row per asset; weights add up to 1; covariance symmetric,
positive semidefinite
```

**What the worked example means:** This calculation determines the overall performance profile of an investment portfolio by combining individual asset weights, their expected returns, and the covariance structure among them. The expected return is computed as the weighted sum of individual asset returns, resulting in 0.12. The total risk is measured by the variance, calculated using the quadratic form of the weight vector and the covariance matrix, which yields 0.03168. The standard deviation, representing the volatility or typical fluctuation range, is derived as the square root of this variance, giving 0.177989. This demonstrates how diversification can mitigate risk when assets do not move perfectly together.

### Sharpe ratio, `sharpe_ratio`

Mean excess return per unit of total risk, from a return series (sample standard deviation, n - 1); annualized by sqrt(periods per year).

$$S = \frac{\bar{R} - r_f}{s_R}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `returns` | in | periodic returns (decimals) | , |
| `risk_free` | in | risk-free rate per period (decimal, default 0) (optional) | , |
| `periods_per_year` | in | periods per year for annualizing (12 monthly, 252 daily; default 1) (optional) | , |
| `sharpe` | out | (mean - rf) / sd per period | , |
| `annualized_sharpe` | out | sharpe x sqrt(periods per year) | , |
| `mean_excess_return` | out | mean - rf | , |
| `std_dev` | out | sample standard deviation | , |

**Applies when:** at least 2 returns, not all equal; returns between -100% and 100%.

```
$ economics-finance solve sharpe_ratio "returns=[0.05, 0.02, -0.01, 0.04]" risk_free=0.005
sharpe = 0.755928946
annualized_sharpe = 0.755928946
mean_excess_return = 0.02
std_dev = 0.02645751311
  by: Sharpe ratio  (S = (mean(R) - rf) / s(R), s with n - 1)
  risk_free = 0.005
  assumes: at least 2 returns, not all equal; returns between -100% and 100%
```

**What the worked example means:** This calculation determines the risk-adjusted performance of an investment by comparing its average return to a benchmark. The mean excess return, calculated as the difference between the portfolio's average return and the risk-free rate, stands at 0.02. This figure is divided by the sample standard deviation of returns, which measures total volatility and equals 0.0264575. The resulting per-period Sharpe ratio is 0.755929, indicating how much extra return was earned for each unit of risk taken. To facilitate comparison across different time horizons, this value is scaled by the square root of the number of periods in a year, yielding an annualized Sharpe ratio of 0.755929. A higher value suggests that the investment provided better compensation for the volatility assumed.

### Sortino ratio, `sortino_ratio`

Mean return above a target per unit of downside deviation sqrt(mean(min(0, R - T)^2)) over ALL observations.

$$\text{Sortino} = \frac{\bar{R} - T}{\sqrt{\frac{1}{n}\sum_i \min(0, R_i - T)^2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `returns` | in | periodic returns (decimals) | , |
| `target` | in | target (minimum acceptable) return per period (default 0) (optional) | , |
| `sortino` | out | (mean - T) / downside deviation | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `downside_deviation` | out | sqrt(sum min(0, R - T)^2 / n) | , |
| `mean_return` | out | mean return | , |

**Applies when:** at least 2 returns; at least one return below the target.

```
$ economics-finance solve sortino_ratio "returns=[0.05, 0.02, -0.01, 0.04, -0.03]"
sortino = 0.9899494937
downside_deviation = 0.01414213562
mean_return = 0.014
  by: Sortino ratio  (Sortino = (mean(R) - T) / sqrt((1/n) sum min(0, R - T)^2))
  assumes: at least 2 returns; at least one return below the target
```

**What the worked example means:** This calculation determines the Sortino ratio, which measures risk-adjusted performance by focusing exclusively on downside volatility. It is derived from the periodic returns provided and a specified target return. The result, 0.989949, indicates how much excess return was generated above the target for each unit of downside deviation. A higher value suggests better compensation for the specific risk of falling below the acceptable threshold.

### Tangency (maximum Sharpe ratio) portfolio, `tangency_portfolio`

Weights of the risky portfolio with the highest Sharpe ratio (short sales allowed): w proportional to Sigma^-1 (mu - rf).

$$w^* = \frac{\Sigma^{-1}(\mu - r_f \mathbf{1})}{\mathbf{1}^\top \Sigma^{-1}(\mu - r_f \mathbf{1})}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `expected_returns` | in | expected returns of the assets (decimals) | , |
| `covariance` | in | covariance matrix of the returns | , |
| `risk_free` | in | risk-free rate (decimal) | , |
| `weights` | out | tangency weights (list, add up to 1) | , |
| `expected_return` | out | portfolio expected return | , |
| `std_dev` | out | portfolio sd | , |
| `sharpe` | out | (E - rf) / sd | , |

**Applies when:** covariance positive definite; 1' Sigma^-1 (mu - rf) > 0.

```
$ economics-finance solve tangency_portfolio "expected_returns=[0.1, 0.15]" "covariance=[[0.04,
0.006], [0.006, 0.09]]" risk_free=0.03
weights = [0.5602409638554217,0.4397590361445783]
expected_return = 0.1219879518
std_dev = 0.181428142
sharpe = 0.5070214069
  by: Tangency (maximum Sharpe ratio) portfolio  (z = Sigma^-1 (mu - rf) by Cholesky, w = z /
sum(z))
  risk_free = 0.03
  assumes: covariance positive definite; 1' Sigma^-1 (mu - rf) > 0
```

**What the worked example means:** This calculation determines the optimal risky portfolio that maximizes the reward-to-risk trade-off. Using the provided expected returns, covariance matrix, and risk-free rate, it computes asset weights that sum to one. The resulting portfolio has an expected return of 0.121988, a standard deviation of

0.181428, and achieves a Sharpe ratio of 0.507021, representing the highest possible efficiency among all risky combinations.

**Treynor ratio and Jensen's alpha, `treynor_jensen`**

Excess return per unit of beta (Treynor) and the return above the CAPM prediction (Jensen's alpha), from paired portfolio and market returns and a constant risk-free rate per period.

$$T = \frac{\bar{R}_p - r_f}{\beta_p}, \quad \alpha_J = \bar{R}_p - [r_f + \beta_p(\bar{R}_m - r_f)]$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `portfolio_returns` | in | portfolio returns (decimals) per period | , |
| `market_returns` | in | market returns in the same periods | , |
| `risk_free` | in | risk-free rate per period (decimal, default 0) (optional) | , |
| `beta` | out | beta of the portfolio | , |
| `treynor` | out | (mean Rp - rf) / beta | , |
| `jensen_alpha` | out | mean Rp - [rf + beta (mean Rm - rf)] | , |
| `mean_portfolio_return` | out | mean Rp | , |
| `mean_market_return` | out | mean Rm | , |

**Applies when:** same number of observations, at least 3; market returns not constant; beta not 0.

```
$ economics-finance solve treynor_jensen "portfolio_returns=[0.03, -0.01, 0.04, 0.02]"
"market_returns=[0.02, -0.02, 0.03, 0.01]" risk_free=0.002
beta = 1
treynor = 0.018
jensen_alpha = 0.01
mean_portfolio_return = 0.02
mean_market_return = 0.01
  by: Treynor ratio and Jensen's alpha  (beta = S_pm / S_mm; Treynor = (mean Rp - rf)/beta; alpha
= mean Rp - rf - beta (mean Rm - rf))
  risk_free = 0.002
  assumes: same number of observations, at least 3; market returns not constant; beta not 0
```

**What the worked example means:** This calculation determines the portfolio's sensitivity to market movements, known as beta, which is 1. Using this measure of systematic risk gives the Treynor ratio at 0.018, representing excess return per unit of beta. Additionally, Jensen's alpha is found to be 0.01, indicating the portion of the portfolio's return that exceeds what would be predicted by the Capital Asset Pricing Model given its risk level.

**Two-asset portfolio return and risk, `two_asset_portfolio`**

Expected return and standard deviation of a portfolio of two assets from the weight, means, standard deviations and correlation.

$$\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\rho\,\sigma_1\sigma_2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `weight_1` | in | weight of asset 1 (asset 2 gets 1 - w1) | , |
| `return_1` | in | expected return of asset 1 (decimal) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `return_2` | in | expected return of asset 2 (decimal) | , |
| `sd_1` | in | standard deviation of asset 1 (decimal) | , |
| `sd_2` | in | standard deviation of asset 2 (decimal) | , |
| `correlation` | in | correlation between the two assets (-1..1) | , |
| `expected_return` | out | w1 r1 + w2 r2 | , |
| `std_dev` | out | portfolio standard deviation | , |
| `variance` | out | portfolio variance | , |
| `weight_2` | out | 1 - w1 | , |

**Applies when:** -1 <= correlation <= 1; standard deviations >= 0.

```
$ economics-finance solve two_asset_portfolio weight_1=0.6 return_1=0.1 return_2=0.15 sd_1=0.2
sd_2=0.3 correlation=0.1
expected_return = 0.12
std_dev = 0.1779887637
variance = 0.03168
weight_2 = 0.4
  by: Two-asset portfolio return and risk  (var = w1^2 s1^2 + w2^2 s2^2 + 2 w1 w2 rho s1 s2)
  weight_1 = 0.6
  return_1 = 0.1
  return_2 = 0.15
  sd_1 = 0.2
  sd_2 = 0.3
  correlation = 0.1
  assumes: -1 <= correlation <= 1; standard deviations >= 0
```

**What the worked example means:** This calculation determines the risk and return profile for a two-asset portfolio. Using a weight of 0.6 for the first asset, the second asset receives the remaining weight of 0.4. The resulting expected return is 0.12, derived from the weighted average of individual returns. The portfolio variance is computed as 0.03168, which translates to a standard deviation of 0.177989. This metric reflects the combined volatility influenced by the correlation of 0.1 between the assets.

## Risk measures (4)

### Historical value at risk and expected shortfall, `historical_var`

VaR and CVaR (expected shortfall) by historical simulation: with k = ceil(n (1 - confidence)), VaR is minus the k-th smallest return and CVaR minus the mean of the k smallest, times the portfolio value.

$$\mathrm{VaR}_c = -R_{(k)}, \quad \mathrm{CVaR}_c = -\frac{1}{k}\sum_{i=1}^{k} R_{(i)}, \quad k = \lceil n(1-c) \rceil$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `returns` | in | historical returns (decimals) | , |
| `confidence` | in | confidence level (0.95 = 95%) | , |
| `portfolio_value` | in | portfolio value (default 1: VaR as a fraction) (optional) | , |
| `var` | out | value at risk (a positive number is a loss) | , |
| `cvar` | out | expected shortfall (mean loss beyond VaR) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `tail_observations` | out | k, the number of returns in the tail | , |

**Applies when:** at least 2 returns; 0.5 < confidence < 1; portfolio_value > 0.

```
$ economics-finance solve historical_var "returns=[-0.02, 0.01, -0.035, 0.004, 0.012, -0.01,
0.02, -0.005, 0.007, 0.001, -0.015]" confidence=0.9
var = 0.02
cvar = 0.0275
tail_observations = 2
  by: Historical value at risk and expected shortfall  (k = ceil(n (1 - c)); VaR = -R_(k); CVaR =
-(1/k) sum_{i<=k} R_(i))
  confidence = 0.9
  assumes: at least 2 returns; 0.5 < confidence < 1; portfolio_value > 0
```

**What the worked example means:** The historical value at risk is 0.02 and the expected shortfall is 0.0275, based on a confidence level of 0.9.

**Maximum drawdown, `max_drawdown`**

The largest fall from a running peak to a later trough, as a fraction of the peak.

$$\mathrm{MDD} = \max_{t} \left( 1 - \frac{P_t}{\max_{s \le t} P_s} \right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `prices` | in | price or portfolio value series in time order | , |
| `max_drawdown` | out | max over t of 1 - P_t / max_{s<=t} P_s | , |

**Applies when:** prices > 0; at least 2.

```
$ economics-finance solve max_drawdown "prices=[100, 120, 90, 110, 80, 130]"
max_drawdown = 0.3333333333
  by: Maximum drawdown  (running peak; drawdown 1 - P/peak; maximum over time)
  assumes: prices > 0; at least 2
```

**What the worked example means:** Maximum drawdown calculates the largest percentage drop from any historical peak to a subsequent trough within the provided time series. By tracking the highest value reached up to each point in time and comparing it to the current price, the method identifies the worst loss experienced by an investor holding the asset without selling during that period. The result, 0.333333, indicates the severity of this decline relative to the preceding high.

**Parametric (normal) value at risk and expected shortfall, `parametric_var`**

VaR and CVaR of normally distributed returns with a mean and standard deviation per period, scaled to a horizon by mean x h and sd x sqrt(h).

$$\mathrm{VaR} = V(z_c \sigma \sqrt{h} - \mu h), \quad \mathrm{ES} = V \left( \frac{\sigma \sqrt{h}\, \varphi(z_c)}{1 - c} - \mu h \right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `mean` | in | mean return per period (decimal) | , |
| `std_dev` | in | standard deviation of returns per period (decimal) | , |
| `confidence` | in | confidence level (0.95 = 95%) | , |
| `horizon` | in | horizon in periods (default 1) (optional) | , |
| `portfolio_value` | in | portfolio value (default 1) (optional) | , |
| `var` | out | value at risk V (z sd sqrt(h) - mean h) | , |
| `cvar` | out | expected shortfall V (sd sqrt(h) phi(z)/(1 - c) - mean h) | , |
| `z` | out | standard normal quantile at the confidence level | , |

**Applies when:** 0 < std_dev < 5; 0.5 < confidence < 1; horizon > 0; returns normally distributed.

```
$ economics-finance solve parametric_var mean=0.0005 std_dev=0.012 confidence=0.99 horizon=10
portfolio_value=1000000
var = 83278.69494
cvar = 96137.76866
z = 2.326347874
  by: Parametric (normal) value at risk and expected shortfall  (VaR = V (z_c sigma sqrt(h) - mu
h); ES = V (sigma sqrt(h) phi(z_c) / (1 - c) - mu h))
  mean = 0.0005
  std_dev = 0.012
  confidence = 0.99
  horizon = 10
  portfolio_value = 1000000
  assumes: 0 < std_dev < 5; 0.5 < confidence < 1; horizon > 0; returns normally distributed
```

**What the worked example means:** This calculation estimates the potential loss for a portfolio using normal distribution assumptions. It takes the mean return per period of 0.0005, standard deviation of 0.012, confidence level of 0.99, and horizon of 10 periods to determine that the value at risk is 83278.7 and the expected shortfall is 96137.8. These figures represent the maximum likely loss and average extreme loss, respectively, scaled from the initial portfolio value of 1e+06.

### Portfolio value at risk (variance-covariance), `portfolio_var`

Zero-mean normal VaR of a portfolio from its weights and the covariance matrix of periodic returns, with the undiversified VaR (sum of the assets' own VaRs) and the diversification benefit.

$$\mathrm{VaR} = z_c \sqrt{w^\top \Sigma w}\,\sqrt{h}\,V$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `weights` | in | portfolio weights (fractions of the portfolio value) | , |
| `covariance` | in | covariance matrix of periodic returns | , |
| `confidence` | in | confidence level (0.99 = 99%) | , |
| `portfolio_value` | in | portfolio value (default 1) (optional) | , |
| `horizon` | in | horizon in periods (default 1) (optional) | , |
| `var` | out | z sqrt(w'Sigma w) sqrt(h) V | , |
| `portfolio_sd` | out | sqrt(w'Sigma w) per period | , |
| `undiversified_var` | out | z sqrt(h) V sum \|w_i\| sigma_i | , |
| `diversification_benefit` | out | undiversified - portfolio VaR | , |

**Applies when:** one weight per row of the covariance; covariance symmetric positive semidefinite; 0.5 < confidence < 1.

```
$ economics-finance solve portfolio_var "weights=[0.6, 0.4]" "covariance=[[0.0004, 6e-05], [6e-
05, 0.0009]]" confidence=0.99 portfolio_value=1000000
var = 41406.3782
portfolio_sd = 0.01779887637
undiversified_var = 55832.34898
diversification_benefit = 14425.97078
  by: Portfolio value at risk (variance-covariance)  (VaR = z_c sqrt(w' Sigma w) sqrt(h) V (mean
return taken as 0))
  confidence = 0.99
  portfolio_value = 1000000
  assumes: one weight per row of the covariance; covariance symmetric positive semidefinite; 0.5
< confidence < 1
```

**What the worked example means:** This calculation determines the potential loss for a portfolio over a specific time horizon using the variance-covariance method. It combines asset weights and return covariances to find the portfolio standard deviation, then scales it by the confidence level z-score and total value. The result shows that the diversified portfolio VaR is 41406.4, while the sum of individual asset risks would be 55832.3, highlighting a diversification benefit of 14426.

## Stocks (5)

### Capital asset pricing model, `capm`

Expected return of an asset from its beta: E[R] = rf + beta (E[Rm] - rf) (the security market line).

$$E[R_i] = r_f + \beta_i\,(E[R_m] - r_f)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `risk_free` | in | risk-free rate (decimal) | , |
| `beta` | in | beta of the asset | , |
| `market_return` | in | expected market return (decimal) | , |
| `expected_return` | out | expected (required) return | , |
| `market_risk_premium` | out | rm - rf | , |
| `risk_premium` | out | beta (rm - rf) | , |

**Applies when:** -1 < risk_free < 1; -1 < market_return < 1.

```
$ economics-finance solve capm risk_free=0.03 beta=1.2 market_return=0.09
expected_return = 0.102
market_risk_premium = 0.06
risk_premium = 0.072
  by: Capital asset pricing model  (E[R] = rf + beta (rm - rf))
  risk_free = 0.03
  beta = 1.2
  market_return = 0.09
  assumes: -1 < risk_free < 1; -1 < market_return < 1
```

**What the worked example means:** This calculation determines the required rate of return for an asset using the Capital Asset Pricing Model. Combining the risk-free rate of 0.03 with the asset's systematic risk, measured by a beta of 1.2, multiplied by the market risk premium of 0.06, gives an expected return of 0.102. This figure represents the

minimum return investors should demand to compensate for the non-diversifiable risk associated with holding this specific security.

**Dividend discount model (Gordon growth), `gordon_growth`**

Value of a share whose dividend grows at a constant rate forever: P0 = D1 / (r - g).

$$P_0 = \frac{D_1}{r - g} = \frac{D_0(1+g)}{r - g}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `required_return` | in | required return on equity (decimal) | , |
| `growth` | in | constant dividend growth rate (decimal) | , |
| `dividend` | in | dividend just paid D0 (give this OR next_dividend) (optional) | , |
| `next_dividend` | in | dividend expected one year from now D1 (give this OR dividend) (optional) | , |
| `price` | out | share value P0 = D1 / (r - g) | , |
| `next_dividend` | out | D1 = D0 (1 + g) | , |
| `dividend_yield` | out | D1 / P0 = r - g | , |

**Applies when:** 0 < required_return < 1; growth < required_return; exactly one of dividend (D0), next_dividend (D1).

```
$ economics-finance solve gordon_growth dividend=2 required_return=0.1 growth=0.04
price = 34.66666667
next_dividend = 2.08
dividend_yield = 0.06
  by: Dividend discount model (Gordon growth)  (P0 = D0 (1 + g) / (r - g))
  dividend = 2
  required_return = 0.1
  growth = 0.04
  assumes: 0 < required_return < 1; growth < required_return; exactly one of dividend (D0),
next_dividend (D1)
```

**What the worked example means:** This calculation determines the intrinsic value of a stock using the Gordon Growth Model, which assumes dividends increase at a steady rate forever. Given a required return of 0.1 and a constant growth rate of 0.04, the estimated share price is 34.6667. This figure represents the present value of all future dividend payments, providing a baseline for assessing whether the market price reflects fundamental value.

**Required return implied by the Gordon model, `gordon_implied_return`**

The return investors require given a share price and a constant dividend growth rate: r = D1 / P0 + g.

$$r = \frac{D_1}{P_0} + g$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `price` | in | current share price | , |
| `growth` | in | constant dividend growth rate (decimal) | , |
| `dividend` | in | dividend just paid D0 (give this OR next_dividend) (optional) | , |
| `next_dividend` | in | dividend expected one year from now D1 (give this OR dividend) (optional) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `required_return` | out | implied required return D1/P0 + g | , |
| `dividend_yield` | out | D1 / P0 | , |
| `next_dividend` | out | D1 | , |

**Applies when:** price > 0; exactly one of dividend (D0), next_dividend (D1); dividend > 0.

```
$ economics-finance solve gordon_implied_return price=50 dividend=2 growth=0.04
required_return = 0.0816
dividend_yield = 0.0416
next_dividend = 2.08
  by: Required return implied by the Gordon model  (r = D1 / P0 + g)
  price = 50
  dividend = 2
  growth = 0.04
  assumes: price > 0; exactly one of dividend (D0), next_dividend (D1); dividend > 0
```

**What the worked example means:** This calculation determines the rate of return investors expect from a stock based on its current price and anticipated dividend growth. The implied required return is 0.0816, which combines the expected dividend yield of 0.0416 with the constant growth rate of 0.04. This result helps practitioners assess whether a stock's market price reflects reasonable return expectations given its fundamentals.

**Two-stage dividend discount model, `two_stage_ddm`**

Share value when dividends grow at a high rate for n years and at a stable rate forever after.

$$P_0 = \sum_{k=1}^{n} \frac{D_0(1+g_1)^k}{(1+r)^k} + \frac{D_n(1+g_2)}{(r-g_2)(1+r)^n}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `dividend` | in | dividend just paid D0 | , |
| `high_growth` | in | growth rate in the first stage (decimal) | , |
| `high_growth_years` | in | number of years of high growth | , |
| `stable_growth` | in | growth rate forever after (decimal) | , |
| `required_return` | in | required return on equity (decimal) | , |
| `price` | out | share value | , |
| `terminal_value` | out | value at year n of the dividends after n | , |
| `pv_terminal` | out | present value of the terminal value | , |
| `pv_high_growth_dividends` | out | present value of D1..Dn | , |

**Applies when:** dividend > 0; stable_growth < required_return; high_growth_years a whole number >= 1.

```
$ economics-finance solve two_stage_ddm dividend=2 high_growth=0.15 high_growth_years=5
stable_growth=0.04 required_return=0.1
price = 54.74416166
terminal_value = 69.72704917
pv_terminal = 43.29501162
pv_high_growth_dividends = 11.44915004
  by: Two-stage dividend discount model  (P0 = sum_{k=1}^{n} D0 (1+g1)^k / (1+r)^k + [D_n (1 +
g2)/(r - g2)] / (1 + r)^n)
  dividend = 2
  high_growth = 0.15
  stable_growth = 0.04
  required_return = 0.1
  assumes: dividend > 0; stable_growth < required_return; high_growth_years a whole number >= 1
```

**What the worked example means:** This calculation determines the intrinsic stock price by summing the present value of dividends growing at a high rate for the first few years with the present value of all future dividends assuming stable growth. Using a recent dividend of 2, a high growth phase lasting 5 years, and a long-term growth rate of 0.04, the model yields a total value of 54.7442. The terminal component alone contributes 43.295 to this figure.

### Weighted average cost of capital, `wacc`

WACC = E/V re + D/V rd (1 - t): the after-tax cost of a firm's capital at market-value weights.

$$\mathrm{WACC} = \frac{E}{V} r_e + \frac{D}{V} r_d (1 - t)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `equity_value` | in | market value of equity | , |
| `debt_value` | in | market value of debt | , |
| `cost_of_equity` | in | cost of equity (decimal) | , |
| `cost_of_debt` | in | pre-tax cost of debt (decimal) | , |
| `tax_rate` | in | corporate tax rate (decimal) | , |
| `wacc` | out | weighted average cost of capital | , |
| `after_tax_cost_of_debt` | out | rd (1 - t) | , |
| `equity_weight` | out | E / V | , |
| `debt_weight` | out | D / V | , |

**Applies when:** equity_value, debt_value >= 0 and not both 0; 0 <= tax_rate < 0.99.

```
$ economics-finance solve wacc equity_value=600 debt_value=400 cost_of_equity=0.12
cost_of_debt=0.06 tax_rate=0.25
wacc = 0.09
after_tax_cost_of_debt = 0.045
equity_weight = 0.6
debt_weight = 0.4
  by: Weighted average cost of capital  (WACC = E/V re + D/V rd (1 - t))
  equity_value = 600
  debt_value = 400
  cost_of_equity = 0.12
  cost_of_debt = 0.06
  tax_rate = 0.25
  assumes: equity_value, debt_value >= 0 and not both 0; 0 <= tax_rate < 0.99
```

**What the worked example means:** This calculation determines the firm's blended cost of financing by combining the market value weights of its equity and debt with their respective costs. The result, 0.09, represents the minimum return required to satisfy both shareholders and lenders after accounting for tax shields on interest payments. Specifically, it uses an equity weight of 0.6 applied to a cost of equity of 0.12, plus a debt weight of 0.4 applied to an after-tax cost of debt of 0.045. This figure serves as the primary discount rate for evaluating new projects or estimating enterprise value.

## Stocks and options (7)

### Binomial option price (Cox-Ross-Rubinstein), `binomial_option`

European call or put price on an n-step Cox-Ross-Rubinstein tree (u = e^(sigma sqrt(dt)), d = 1/u, with the delta of the first step.

$$u = e^{\sigma\sqrt{\Delta t}},\ d = 1/u,\ p = \frac{e^{(r-q)\Delta t} - d}{u - d},\ V = e^{-r\Delta t}[pV_u + (1-p)V_d]$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `spot` | in | price of the underlying now | , |
| `strike` | in | strike (exercise) price | , |
| `time` | in | time to expiry in years | , |
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `volatility` | in | volatility of the underlying, annual (decimal) | , |
| `steps` | in | number of time steps of the tree | , |
| `option_type` | in | call (default) or put (optional) | , |
| `dividend_yield` | in | continuous dividend yield (decimal, default 0) (optional) | , |
| `price` | out | option price at time 0 | , |
| `delta` | out | (V_u - V_d) / (S u - S d) | , |
| `up_factor` | out | u | , |
| `probability_up` | out | risk-neutral p = (e^((r-q)dt) - d)/(u - d) | , |

**Applies when:** European exercise; 1 <= steps <= 1000; 0 < p < 1.

```
$ economics-finance solve binomial_option spot=100 strike=100 time=1 rate=0.05 volatility=0.2
steps=3
price = 11.04387109
delta = 0.6140855606
up_factor = 1.122400902
probability_up = 0.5437765964
  by: Binomial option price (Cox-Ross-Rubinstein)  (backward induction V = e^(-r dt) (p V_up + (1
- p) V_down) from the payoffs at expiry)
  spot = 100
  strike = 100
  time = 1
  rate = 0.05
  volatility = 0.2
  assumes: European exercise; 1 <= steps <= 1000; 0 < p < 1
```

**What the worked example means:** This calculation determines the fair value of a European option using a Cox-Ross-Rubinstein binomial tree. It constructs a lattice of potential underlying asset prices over 3 time steps, starting from the current spot price of 100. The model computes an up factor of 1.1224 and a risk-neutral probability of moving up of 0.543777. By working backward from the expiration payoffs, discounting at the risk-free rate of 0.05, it arrives at a present value of 11.0439. The resulting delta of 0.614086 indicates the sensitivity of the option's value to changes in the underlying asset's price.

### Black-Scholes Greeks, `bs_greeks`

Delta, gamma, vega, theta and rho of European calls and puts (Black-Scholes-Merton, continuous dividend yield). Vega per 1.00 of volatility, theta per year, rho per 1.00 of rate.

$$\Delta_C = e^{-qT} N(d_1),\ \Gamma = \frac{e^{-qT}\varphi(d_1)}{S\sigma\sqrt{T}},\ \nu = Se^{-qT}\varphi(d_1)\sqrt{T}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `spot` | in | price of the underlying now | , |
| `strike` | in | strike (exercise) price | , |
| `time` | in | time to expiry in years | , |
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `volatility` | in | volatility of the underlying, annual (decimal) | , |
| `dividend_yield` | in | continuous dividend yield (decimal, default 0) (optional) | , |
| `call_delta` | out | dC/dS | , |
| `put_delta` | out | dP/dS | , |
| `gamma` | out | d2V/dS2 (call = put) | , |
| `vega` | out | dV/dsigma (call = put) | , |
| `call_theta` | out | -dC/dT per year | , |
| `put_theta` | out | -dP/dT per year | , |
| `call_rho` | out | dC/dr | , |
| `put_rho` | out | dP/dr | , |

**Applies when:** spot, strike, time > 0; 0 < volatility < 5.

```
$ economics-finance solve bs_greeks spot=100 strike=100 time=1 rate=0.05 volatility=0.2
call_delta = 0.6368306512
put_delta = -0.3631693488
gamma = 0.01876201735
vega = 37.52403469
call_theta = -6.414027546
put_theta = -1.657880424
call_rho = 53.23248155
put_rho = -41.8904609
  by: Black-Scholes Greeks  (closed-form Greeks of the Black-Scholes-Merton formula)
  spot = 100
  strike = 100
  time = 1
  rate = 0.05
  volatility = 0.2
  assumes: spot, strike, time > 0; 0 < volatility < 5
```

In the worked example, spot = 100, strike = 100, time = 1, rate = 0.05, volatility = 0.2 go through the equation above and give call_delta = 0.6368306512, put_delta = -0.3631693488, gamma = 0.01876201735, vega = 37.52403469. It applies only when spot, strike, time > 0; 0 < volatility < 5: outside that it says what is wrong instead of returning a number.

**Black-Scholes option prices, `black_scholes`**

Prices of European call and put options on an asset with a continuous dividend yield (Black-Scholes-Merton).

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2), \quad d_1 = \frac{\ln(S/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `spot` | in | price of the underlying now | , |
| `strike` | in | strike (exercise) price | , |
| `time` | in | time to expiry in years | , |
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `volatility` | in | volatility of the underlying, annual (decimal) | , |
| `dividend_yield` | in | continuous dividend yield (decimal, default 0) (optional) | , |
| `call` | out | European call price | , |
| `put` | out | European put price | , |
| `d1` | out | d1 | , |
| `d2` | out | d2 = d1 - sigma sqrt(T) | , |

**Applies when:** spot, strike, time > 0; 0 < volatility < 5; European exercise; lognormal prices.

```
$ economics-finance solve black_scholes spot=100 strike=100 time=1 rate=0.05 volatility=0.2
call = 10.45058357
put = 5.573526022
d1 = 0.35
d2 = 0.15
  by: Black-Scholes option prices  (C = S e^(-qT) N(d1) - K e^(-rT) N(d2); P = K e^(-rT) N(-d2) -
S e^(-qT) N(-d1))
  spot = 100
  strike = 100
  time = 1
  rate = 0.05
  volatility = 0.2
  assumes: spot, strike, time > 0; 0 < volatility < 5; European exercise; lognormal prices
```

**What the worked example means:** This calculation determines the fair theoretical value of European call and put options using the Black-Scholes-Merton framework. It incorporates the current underlying asset price, exercise price, time to expiration, risk-free interest rate, volatility, and continuous dividend yield. The resulting call price of 10.4506 represents the premium paid for the right to buy the asset at 100, while the put price of 5.57353 reflects the cost of the right to sell it at that same level.

**Forward price with cost of carry, `forward_price`**

No-arbitrage forward price of an asset with a continuous income yield q and storage cost u: S e^((r + u - q) T).

$$F_0 = S_0\, e^{(r+u-q)T}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `spot` | in | spot price now | , |
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `time` | in | time to delivery in years | , |
| `dividend_yield` | in | continuous income or convenience yield (decimal, default 0) (optional) | , |
| `storage_cost` | in | continuous storage cost rate (decimal, default 0) (optional) | , |
| `forward` | out | forward price | , |
| `cost_of_carry` | out | r + u - q | , |

**Applies when:** spot > 0; time >= 0; rates as continuously compounded decimals.

```
$ economics-finance solve forward_price spot=100 rate=0.05 time=0.5 dividend_yield=0.02
forward = 101.5113065
cost_of_carry = 0.03
  by: Forward price with cost of carry  (F = S exp((r + u - q) T))
  spot = 100
  rate = 0.05
  time = 0.5
  dividend_yield = 0.02
  assumes: spot > 0; time >= 0; rates as continuously compounded decimals
```

**What the worked example means:** This calculation determines the fair forward price for an asset that incurs storage costs and generates income yields. By applying the cost of carry model, it equates the present value of buying and holding the asset with the discounted forward delivery price to ensure zero initial value for the contract. The resulting forward price of 101.511 reflects a net carrying cost of 0.03 applied over a period of 0.5 years, starting

from a current spot price of 100. This figure represents the theoretical price agreed upon today for future delivery, assuming continuous compounding and no arbitrage opportunities.

**Forward price with known dividends, `forward_price_dividends`**

Forward price of a stock paying known cash dividends before delivery: F = (S - PV(dividends)) e^(rT).

$$F_0 = \left( S_0 - \sum_i D_i e^{-r t_i} \right) e^{rT}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `spot` | in | stock price now | , |
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `time` | in | time to delivery in years | , |
| `dividends` | in | cash dividends (list) | , |
| `dividend_times` | in | time of each dividend in years (list) | , |
| `forward` | out | forward price | , |
| `pv_dividends` | out | present value of the dividends | , |

**Applies when:** 0 < each dividend time <= time; PV(dividends) < spot.

```
$ economics-finance solve forward_price_dividends spot=50 rate=0.05 time=1 "dividends=[0.75,
0.75]" "dividend_times=[0.25, 0.75]"
forward = 51.02546198
pv_dividends = 1.463079164
  by: Forward price with known dividends  (F = (S - sum D_i e^(-r t_i)) e^(rT))
  spot = 50
  rate = 0.05
  time = 1
  assumes: 0 < each dividend time <= time; PV(dividends) < spot
```

**What the worked example means:** This calculation determines the fair forward price for a stock that pays known cash dividends before delivery. It starts with the current stock price, 50, and subtracts the present value of all expected dividends, calculated as 1.46308. This adjusted amount represents the cost of carrying the stock without the benefit of the dividends. The result is then compounded at the risk-free rate, 0.05, over the time to delivery, 1, yielding the final forward price of 51.0255. This ensures no arbitrage opportunity exists between buying the stock now versus entering the forward contract.

**Implied volatility, `implied_volatility`**

The volatility at which the Black-Scholes price of a European call or put equals its market price.

$$C_{BS}(\sigma_{imp}) = C_{market}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `option_price` | in | market price of the option | , |
| `spot` | in | price of the underlying now | , |
| `strike` | in | strike (exercise) price | , |
| `time` | in | time to expiry in years | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `dividend_yield` | in | continuous dividend yield (decimal, default 0) (optional) | , |
| `option_type` | in | call (default) or put (optional) | , |
| `volatility` | out | implied volatility, annual | , |
| `vega` | out | vega at that volatility | , |

**Applies when:** price within the no-arbitrage bounds; implied volatility between 0.01% and 500%; not too far from the money.

```
$ economics-finance solve implied_volatility option_price=10.45 spot=100 strike=100 time=1
rate=0.05
volatility = 0.199984448
vega = 37.52388148
  by: Implied volatility  (bisection on sigma (the price rises with sigma, so the implied
volatility is unique))
  option_price = 10.45
  spot = 100
  strike = 100
  time = 1
  rate = 0.05
  assumes: price within the no-arbitrage bounds; implied volatility between 0.01% and 500%; not
too far from the money
```

**What the worked example means:** This calculation determines the implied volatility, which is the specific annualized standard deviation of returns that makes the Black-Scholes theoretical price match the observed market price of 10.45. Solving for this variable using bisection isolates the market's expectation of future uncertainty for the underlying asset priced at 100 with a strike of 100 and time to expiry of 1 years. The resulting value of 0.199984 allows for standardized comparison across different strikes and maturities, while the associated vega of 37.5239 indicates how sensitive the option price is to changes in this volatility estimate.

**Put-call parity, `put_call_parity`**

The price of a European put from the call with the same strike and expiry: P = C - S e^(-qT) + K e^(-rT).

$$C + Ke^{-rT} = P + Se^{-qT}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `call_price` | in | price of the European call | , |
| `spot` | in | price of the underlying now | , |
| `strike` | in | strike (exercise) price | , |
| `time` | in | time to expiry in years | , |
| `rate` | in | risk-free rate, continuously compounded (decimal) | , |
| `dividend_yield` | in | continuous dividend yield (decimal, default 0) (optional) | , |
| `put_price` | out | European put price | , |
| `pv_strike` | out | K e^(-rT) | , |
| `pv_spot` | out | S e^(-qT) | , |

**Applies when:** European options, same strike and expiry; call within the no-arbitrage bounds.

```
$ economics-finance solve put_call_parity call_price=10.45 spot=100 strike=100 time=1 rate=0.05
put_price = 5.57294245
pv_strike = 95.12294245
pv_spot = 100
  by: Put-call parity  (P = C - S e^(-qT) + K e^(-rT))
  call_price = 10.45
  spot = 100
  strike = 100
  time = 1
  rate = 0.05
  assumes: European options, same strike and expiry; call within the no-arbitrage bounds
```

**What the worked example means:** This calculation determines the fair price of a European put option using put-call parity. Taking the market price of the corresponding call, 10.45, and adjusting for the present values of the underlying asset and the strike price gives a put price of 5.57294. The term 100 represents the current spot price discounted by the dividend yield, while 95.1229 reflects the strike price discounted by the risk-free rate. This relationship ensures that no arbitrage opportunities exist between the call, the put, and the underlying assets under the assumption of European exercise.

## Time value of money (15)

### Compound annual growth rate, `cagr`

The constant annual growth rate that takes a beginning value to an ending value over a number of years.

$$\mathrm{CAGR} = \left(\frac{V_{end}}{V_{begin}}\right)^{1/t} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `begin_value` | in | value at the start | , |
| `end_value` | in | value at the end | , |
| `years` | in | number of years | , |
| `cagr` | out | compound annual growth rate (end/begin)^(1/t) - 1 | , |
| `total_growth` | out | end/begin - 1 | , |

**Applies when:** begin_value > 0; end_value > 0; years > 0.

```
$ economics-finance solve cagr begin_value=1000 end_value=2000 years=5
cagr = 0.148698355
total_growth = 1
  by: Compound annual growth rate  (CAGR = (end/begin)^(1/t) - 1)
  begin_value = 1000
  end_value = 2000
  years = 5
  assumes: begin_value > 0; end_value > 0; years > 0
```

**What the worked example means:** This calculation determines the constant annual growth rate that turns a beginning value of 1000 into an ending value of 2000 over 5 years. The resulting compound annual growth rate is 0.148698, while the overall change in value during this period amounts to 1. This metric smooths out volatility to provide a single rate for comparing performance across different time horizons.

### Compound interest, `compound_interest`

Future amount of a sum invested at a nominal annual rate compounded m times a year: A = P (1 + r/m)^(m t).

$$A = P\left(1 + \frac{r}{m}\right)^{mt}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount invested now | , |
| `rate` | in | nominal annual interest rate (decimal; 5% = 0.05) | , |
| `years` | in | time in years | , |
| `periods_per_year` | in | compounding periods per year (1 annual, 12 monthly, 365 daily) (optional) | , |
| `amount` | out | future amount A | , |
| `interest` | out | compound interest earned A - P | , |

**Applies when:** principal > 0; -1 < rate < 1; years >= 0; periods_per_year a whole number >= 1.

```
$ economics-finance solve compound_interest principal=1000 rate=0.05 years=2
amount = 1102.5
interest = 102.5
  by: Compound interest  (A = P (1 + r/m)^(m t))
  principal = 1000
  rate = 0.05
  years = 2
  assumes: principal > 0; -1 < rate < 1; years >= 0; periods_per_year a whole number >= 1
```

**What the worked example means:** An initial investment of 1000 grows to a future amount of 1102.5, earning 102.5 in compound interest.

**Continuous compounding, `continuous_compounding`**

Future amount of a sum growing at a continuously compounded annual rate: A = P e^(r t).

$$A = Pe^{rt}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount invested now | , |
| `rate` | in | continuously compounded annual rate (decimal) | , |
| `years` | in | time in years | , |
| `amount` | out | future amount P e^(r t) | , |
| `interest` | out | interest earned | , |

**Applies when:** principal > 0; -1 < rate < 1; years >= 0.

```
$ economics-finance solve continuous_compounding principal=1000 rate=0.05 years=10
amount = 1648.721271
interest = 648.7212707
  by: Continuous compounding  (A = P exp(r t))
  principal = 1000
  rate = 0.05
  years = 10
  assumes: principal > 0; -1 < rate < 1; years >= 0
```

**What the worked example means:** This calculation determines the future value of an investment using continuous compounding. Starting with a principal of 1000, applying a continuously compounded annual rate of 0.05 over a period of 10, the total accumulated amount reaches 1648.72. The interest earned during this time is 648.721, representing the growth beyond the initial sum.

**Doubling time, `doubling_time`**

How many periods an amount compounding at r per period takes to double, with the rule-of-72 estimate.

$$n = \frac{\ln 2}{\ln(1 + r)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | interest or growth rate per period (decimal) | , |
| `periods` | out | exact doubling time ln 2 / ln(1 + r) | , |
| `rule_of_72` | out | rule-of-72 estimate 72 / (100 r) | , |

**Applies when:** 0 < rate < 1.

```
$ economics-finance solve doubling_time rate=0.06
periods = 11.89566105
rule_of_72 = 12
  by: Doubling time  (n = ln 2 / ln(1 + r))
  rate = 0.06
  assumes: 0 < rate < 1
```

**What the worked example means:** This calculation determines how many periods it takes for an amount to double when growing at a constant rate of 0.06 per period. The exact doubling time is computed as 11.8957 periods, while the rule-of-72 provides a quick approximation of 12. These values help quantify the impact of compound growth over time.

**Effective annual rate, `effective_annual_rate`**

The effective annual rate (annual percentage yield) of a nominal rate compounded m times a year.

$$\mathrm{EAR} = \left(1 + \frac{r}{m}\right)^{m} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `rate` | in | nominal annual rate (decimal) | , |
| `periods_per_year` | in | compounding periods per year | , |
| `effective_rate` | out | effective annual rate (1 + r/m)^m - 1 | , |
| `periodic_rate` | out | rate per period r/m | , |

**Applies when:** -1 < rate < 1; periods_per_year a whole number >= 1.

```
$ economics-finance solve effective_annual_rate rate=0.06 periods_per_year=12
effective_rate = 0.06167781186
periodic_rate = 0.005
  by: Effective annual rate  (EAR = (1 + r/m)^m - 1)
  rate = 0.06
  assumes: -1 < rate < 1; periods_per_year a whole number >= 1
```

**What the worked example means:** This calculation determines the true annual return or cost by converting a nominal interest rate into an effective annual rate. Given a nominal annual rate of 0.06 and compounding occurring 12 times each year, the periodic rate applied in each interval is 0.005. The resulting effective annual rate is 0.0616778, which represents the actual yield after accounting for the effects of intra-year compounding, allowing for accurate comparison with other financial instruments that may have different compounding frequencies.

**Future value of an annuity, `future_value_annuity`**

The accumulated value after n periods of equal payments, ordinary (end of period) or due (beginning).

$$FV = PMT\,\frac{(1+r)^{n}-1}{r}\;(\times(1+r)\text{ if due})$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `payment` | in | payment per period | , |
| `rate` | in | interest rate per period (decimal) | , |
| `periods` | in | number of payments | , |
| `timing` | in | 'ordinary' (end of each period, default) or 'due' (beginning) (optional) | , |
| `future_value` | out | accumulated value at the end of period n | , |

**Applies when:** -1 < rate < 1; periods a whole number >= 1; timing ordinary or due.

```
$ economics-finance solve future_value_annuity payment=100 rate=0.05 periods=10
future_value = 1257.789254
  by: Future value of an annuity  (FV = PMT ((1 + r)^n - 1)/r, times (1 + r) for an annuity due
(PMT n when r = 0))
  payment = 100
  rate = 0.05
  periods = 10
  assumes: -1 < rate < 1; periods a whole number >= 1; timing ordinary or due
```

**What the worked example means:** The future value of the annuity is 1257.79, calculated from a payment of 100 at a rate of 0.05 over 10 periods.

**Interest rate of an annuity, `annuity_rate`**

The rate per period at which n equal payments are worth a given present value (bisection; the rate is unique because the present value falls as the rate rises).

$$PV = PMT\,\frac{1-(1+r)^{-n}}{r}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `present_value` | in | present value (amount borrowed or price paid) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `payment` | in | payment per period | , |
| `periods` | in | number of payments | , |
| `timing` | in | 'ordinary' (end of each period, default) or 'due' (beginning) (optional) | , |
| `rate` | out | interest rate per period | , |

**Applies when:** present_value > 0; payment > 0; periods a whole number >= 1; the rate lies between -99% and 99%.

```
$ economics-finance solve annuity_rate present_value=1000 payment=263.8 periods=4
rate = 0.02184401908
  by: Interest rate of an annuity  (bisection on PMT (1 - (1 + r)^-n)/r = PV)
  present_value = 1000
  payment = 263.8
  periods = 4
  assumes: present_value > 0; payment > 0; periods a whole number >= 1; the rate lies between
-99% and 99%
```

**What the worked example means:** This calculation determines the implicit interest rate per period that makes the present value of a series of equal payments equal to the initial amount invested or borrowed. Given a present value of 1000 and periodic payments of 263.8 over 4 periods, the method solves for the unique rate where the discounted cash flows sum to the principal. The resulting rate of 0.021844 represents the true cost of financing or yield on the investment stream under these conditions.

**Nominal rate from an effective rate, `nominal_rate`**

The nominal annual rate (APR) compounded m times a year that gives a stated effective annual rate.

$$r = m\left[(1+\mathrm{EAR})^{1/m} - 1\right]$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `effective_rate` | in | effective annual rate (decimal) | , |
| `periods_per_year` | in | compounding periods per year | , |
| `nominal_rate` | out | nominal annual rate m((1 + EAR)^(1/m) - 1) | , |
| `periodic_rate` | out | rate per period | , |

**Applies when:** -1 < effective_rate < 1; periods_per_year a whole number >= 1.

```
$ economics-finance solve nominal_rate effective_rate=0.08 periods_per_year=12
nominal_rate = 0.07720836132
periodic_rate = 0.00643403011
  by: Nominal rate from an effective rate  (r = m ((1 + EAR)^(1/m) - 1))
  effective_rate = 0.08
  assumes: -1 < effective_rate < 1; periods_per_year a whole number >= 1
```

**What the worked example means:** This calculation determines the nominal annual interest rate, often referred to as the APR, that corresponds to a given effective annual rate when compounding occurs multiple times per year. By using the effective annual rate of 0.08 and assuming 12 compounding periods annually, the model computes the equivalent nominal rate of 0.0772084. This conversion allows practitioners to compare quoted nominal rates with actual returns by accounting for the frequency of compounding, ensuring that the stated periodic rate of 0.00643403 accurately reflects the total growth over one year under standard compounding assumptions.

### Present and future value of a growing annuity, `growing_annuity`

n payments at the end of each period, the first C1, each growing at g per period.

$$PV = \frac{C_1}{r - g}\left[1 - \left(\frac{1+g}{1+r}\right)^{n}\right]$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `first_payment` | in | first payment (end of period 1) | , |
| `rate` | in | discount rate per period (decimal) | , |
| `growth` | in | growth rate per period (decimal) | , |
| `periods` | in | number of payments | , |
| `present_value` | out | value today | , |
| `future_value` | out | value at the end of period n | , |

**Applies when:** -1 < rate < 1; -1 < growth < 1; periods a whole number >= 1.

```
$ economics-finance solve growing_annuity first_payment=1000 rate=0.08 growth=0.03 periods=10
present_value = 7550.133691
future_value = 16300.17236
  by: Present and future value of a growing annuity  (PV = C1/(r - g) (1 - ((1 + g)/(1 + r))^n)
(n C1/(1 + r) when r = g); FV = PV (1 + r)^n)
  first_payment = 1000
  rate = 0.08
  growth = 0.03
  assumes: -1 < rate < 1; -1 < growth < 1; periods a whole number >= 1
```

**What the worked example means:** This calculation determines the current worth and the accumulated total at the end of a term for a series of payments that grow at a constant rate. Using a first payment of 1000, a discount rate of 0.08, a growth rate of 0.03, and 10 periods, the model computes a present value of 7550.13 and a future value of 16300.2. These figures help evaluate investments or loans where cash flows change predictably over a fixed number of periods.

### Present value of a growing perpetuity, `growing_perpetuity`

The value today of payments that start at C one period from now and grow at g per period forever: C / (r - g).

$$PV = \frac{C_1}{r - g}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `first_payment` | in | payment one period from now | , |
| `rate` | in | discount rate per period (decimal) | , |
| `growth` | in | growth rate per period (decimal) | , |
| `present_value` | out | value today C1 / (r - g) | , |

**Applies when:** 0 < rate < 1; growth < rate.

```
$ economics-finance solve growing_perpetuity first_payment=100 rate=0.08 growth=0.03
present_value = 2000
  by: Present value of a growing perpetuity  (PV = C1 / (r - g))
  first_payment = 100
  rate = 0.08
  growth = 0.03
  assumes: 0 < rate < 1; growth < rate
```

**What the worked example means:** This calculation determines the current worth of an infinite series of cash flows that begin one period from now. Using a first payment of 100, a discount rate of 0.08, and a constant growth rate of 0.03, the model computes a present value of 2000. This result represents the lump sum today that is equivalent to receiving payments that grow at 0.03 forever, discounted back at 0.08.

**Present value of a perpetuity, `perpetuity`**

The value today of an equal payment every period forever: PV = C / r (C / r + C if payments start now).

$$PV = \frac{C}{r}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `payment` | in | payment per period | , |
| `rate` | in | discount rate per period (decimal) | , |
| `timing` | in | 'ordinary' (first payment in one period, default) or 'due' (first payment now) (optional) | , |
| `present_value` | out | value today | , |

**Applies when:** 0 < rate < 1.

```
$ economics-finance solve perpetuity payment=100 rate=0.05
present_value = 2000
  by: Present value of a perpetuity  (PV = C / r, times (1 + r) for a perpetuity due)
  payment = 100
  rate = 0.05
  assumes: 0 < rate < 1
```

**What the worked example means:** This calculation determines the current worth of an infinite series of equal payments. Dividing the periodic payment of 100 by the discount rate of 0.05 gives a present value of 2000. This figure represents how much you would need to invest today at that rate to generate those perpetual cash flows, assuming the first payment occurs one period from now.

**Present value of a single sum, `present_value`**

What a single amount received after n periods is worth today at a rate per period: PV = FV / (1 + r)^n.

$$PV = \frac{FV}{(1+r)^n}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `future_value` | in | amount received in the future | , |
| `rate` | in | discount rate per period (decimal) | , |
| `periods` | in | number of periods until the amount is received | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `present_value` | out | value today FV (1 + r)^-n | , |
| `discount_factor` | out | (1 + r)^-n | , |

**Applies when:** -1 < rate < 1; periods >= 0.

```
$ economics-finance solve present_value future_value=1000 rate=0.05 periods=10
present_value = 613.9132535
discount_factor = 0.6139132535
  by: Present value of a single sum  (PV = FV (1 + r)^-n)
  future_value = 1000
  rate = 0.05
  periods = 10
  assumes: -1 < rate < 1; periods >= 0
```

**What the worked example means:** This calculation determines the current worth of a single future amount, known as the future value, by applying a discount rate over a specific number of periods. The result, 613.913, represents what that future sum is equivalent to today given the time value of money. The process involves dividing the future amount by one plus the rate raised to the power of the number of periods, effectively reversing the compounding effect to find today's equivalent value.

**Present value of an annuity, `present_value_annuity`**

The value today of n equal payments, ordinary (end of period) or due (beginning).

$$PV = PMT\,\frac{1-(1+r)^{-n}}{r}\;(\times(1+r)\text{ if due})$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `payment` | in | payment per period | , |
| `rate` | in | interest rate per period (decimal) | , |
| `periods` | in | number of payments | , |
| `timing` | in | 'ordinary' (end of each period, default) or 'due' (beginning) (optional) | , |
| `present_value` | out | value today of the payments | , |

**Applies when:** -1 < rate < 1; periods a whole number >= 1; timing ordinary or due.

```
$ economics-finance solve present_value_annuity payment=100 rate=0.05 periods=10
present_value = 772.1734929
  by: Present value of an annuity  (PV = PMT (1 - (1 + r)^-n)/r, times (1 + r) for an annuity due
(PMT n when r = 0))
  payment = 100
  rate = 0.05
  periods = 10
  assumes: -1 < rate < 1; periods a whole number >= 1; timing ordinary or due
```

**What the worked example means:** The present value of the annuity is 772.173, calculated from a payment of 100 over 10 periods at a rate of 0.05.

**Simple interest, `simple_interest`**

Interest and final amount when interest is earned on the principal only: I = P r t.

$$I = P r t, \quad A = P(1 + rt)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `principal` | in | amount invested or borrowed | , |
| `rate` | in | annual simple interest rate (decimal; 5% = 0.05) | , |
| `years` | in | time in years | , |
| `interest` | out | simple interest earned P r t | , |
| `amount` | out | principal plus interest | , |

**Applies when:** principal > 0; 0 <= rate < 1; years >= 0.

```
$ economics-finance solve simple_interest principal=1000 rate=0.05 years=3
interest = 150
amount = 1150
  by: Simple interest  (I = P r t, A = P + I)
  principal = 1000
  rate = 0.05
  years = 3
  assumes: principal > 0; 0 <= rate < 1; years >= 0
```

**What the worked example means:** This calculation determines the simple interest and total repayment or accumulation. It uses a principal of 1000, an annual rate of 0.05, and a time period of 3. The resulting interest is 150, which when added to the original principal gives a final amount of 1150. This reflects growth where earnings are not reinvested.

### Sinking fund payment, `sinking_fund`

The equal payment per period that accumulates to a target amount after n periods.

$$PMT = \frac{FV\, r}{(1 + r)^n - 1}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `future_value` | in | target amount | , |
| `rate` | in | interest rate per period (decimal) | , |
| `periods` | in | number of payments | , |
| `timing` | in | 'ordinary' (end of each period, default) or 'due' (beginning) (optional) | , |
| `payment` | out | payment per period | , |

**Applies when:** future_value > 0; -1 < rate < 1; periods a whole number >= 1.

```
$ economics-finance solve sinking_fund future_value=10000 rate=0.04 periods=5
payment = 1846.271135
  by: Sinking fund payment  (PMT = FV r / ((1 + r)^n - 1), divided by (1 + r) for payments at the
beginning)
  future_value = 10000
  rate = 0.04
  periods = 5
  assumes: future_value > 0; -1 < rate < 1; periods a whole number >= 1
```

**What the worked example means:** This calculation determines the equal payment per period needed to accumulate a target amount. Given a future value of 10000, an interest rate of 0.04, and 5 periods, the result is a payment of 1846.27. If payments are made at the beginning of each period, this amount is adjusted accordingly.

Questions and reports: support@ai999x.com. AI999X, The Future Is Self-Improving.